



Escuela Técnica Superior de
Ingeniería Informática

TRABAJO FIN DE MÁSTER

Uso de IA para la identificación sistemática de errores en la medida de radiación solar

Realizado por
Alejandro Moreno Guerrero

Para la obtención del título de
Máster Universitario en Lógica, Computación e Inteligencia Artificial

Dirigido por
Miguel Larrañeta Gómez-Caminero
Departamento de Ingeniería Energética - ETSI
José Luis Pro Martín
Departamento de Ciencias de la Computación e Inteligencia Artificial - ETSII

Convocatoria de julio, curso 2024/25

Resumen

Las medidas relativas al recurso solar son imprescindibles tanto para las fases de construcción como de operabilidad de una central solar. Para tomar decisiones acertadas es imprescindible que estas medidas sean precisas y fiables. El problema es que las medidas de radiación solar son particularmente propensas a errores, lo que requiere de inversiones en medidas de control de calidad, siendo una de ellas la comparación de curvas de radiación de sensores redundantes, la cual se realiza de forma manual. Gracias a los avances del aprendizaje profundo, es posible crear modelos que realicen esta tarea de forma automática. En este trabajo se ha utilizado un conjunto de datos de irradiancia normal directa (DNI) de tres sensores de una central solar para la clasificación de las curvas en válidas o inválidas. Los modelos que se han utilizado son: una red *feedforward*, una red neuronal convolucional (CNN), un modelo de transformer de visión (ViT) y un modelo transformer PatchTST; y han sido especialmente adaptados a este tipo de conjunto de datos. Los resultados, aunque varían de unos a otros modelos, muestran un buen desempeño, abriendo paso a la posibilidad de automatizar esta tarea.

Palabras clave: Radiación solar, Deep learning, Errores, DNI, IA

Abstract

Solar resource measurements are essential for both the construction and operational phases of a solar power plant. Accurate and reliable measurements are essential for making sound decisions. The problem is that solar radiation measurements are particularly prone to errors, requiring investments in quality control measures, one of which is the comparison of radiation curves from redundant sensors, which is performed manually. Thanks to advances in deep learning, it is possible to create models that perform this task automatically. This work uses a direct normal irradiance (DNI) dataset from three sensors at a solar power plant to classify the curves as valid or invalid. The models used are a feedforward network, a convolutional neural network (CNN), a vision transformer model (ViT), and a PatchTST transformer model; they have been specially adapted to this type of dataset. The results, although varying from one model to another, show good performance, paving the way for the possibility of automating this task.

Keywords: Solar radiation, Errors, Deep learning, DNI, AI

Índice general

1	Introducción	1
1.1.	Fundamentos de radiación solar	2
1.2.	Medición de la radiación solar	5
1.3.	Avances de la IA en detección de errores	14
1.4.	Objetivos	16
1.5.	Estructura del documento	16
2	Metodología	17
2.1.	Herramientas utilizadas	17
2.2.	Conjunto de datos	19
2.3.	Implementación	20
3	Diseño de modelos	27
3.1.	Red <i>feedforward</i> (FF)	29
3.2.	Red neuronal convolucional (CNN)	31
3.3.	Transformer visual (ViT)	34
3.4.	PatchTST	36
3.5.	Optimización de hiperparámetros	39
4	Dificultades encontradas	41
4.1.	Depuración del conjunto de datos	41
4.2.	Aprendizaje de Pytorch	41
4.3.	Optimización de modelos	42
5	Experimentación	46
5.1.	Resultados	46
5.2.	Análisis de eficacia	47
5.3.	Gráficas de entrenamiento	54
6	Discusión	59
7	Conclusiones y trabajos futuros	61
	Bibliografía	62

Índice de figuras

1.1. Tipos de radiación solar	3
1.2. Instalación termosolar	4
1.3. Placas fotovoltaicas	4
1.4. Ángulos de elevación solar (α) y cenital (ϕ)	5
1.5. Curvas de radiación directa y global	6
1.6. Pirheliómetro	7
1.7. Pinanómetro	7
1.8. Ubicación de los radiómetros dentro de una planta solar	8
1.9. Ejemplo de curva de radiación errónea recogida por el sensor S3 . . .	8
1.10. Ejemplo de curvas de radiación válidas	8
1.11. Ejemplo de error poco evidente	9
1.12. Ejemplo 1 de error de mal alineamiento	10
1.13. Ejemplo 2 de error de mal alineamiento	10
1.14. Ejemplo 1 de error con valores perdidos	11
1.15. Ejemplo 2 de error con valores perdidos	11
1.16. Ejemplo de error múltiple	11
1.17. Ejemplo de error por suciedad	12
1.18. Ejemplo de error con nubosidad	12
1.19. Ejemplo de error anómalo	13
1.20. Ejemplo de error dudoso	13
1.21. Ejemplo de series válidas con día nublado	14
2.1. Tabla Excel con los datos de irradiancia	19
2.2. Árbol de directorios en el que se estructura el conjunto de datos . . .	20
3.1. Esquema de FF	29
3.2. Convolución con <i>padding</i> circular	31
3.3. Esquema general de CNN	32
3.4. Extracción de <i>patches</i> de ViT	34
3.5. Arquitectura de un transformer clasificador	35
3.6. Extracción de <i>patches</i> de PatchTST	38
4.1. Gráficas de entrenamiento de CNN antes de optimizar	44
4.2. Gráficas de entrenamiento de PTST con independencia de canales . .	45

5.1. Número de días inválidos en función del umbral de clasificación . .	49
5.2. Gráficas de entrenamiento de FF	55
5.3. Gráficas de entrenamiento de CNN	56
5.4. Gráficas de entrenamiento de ViT	57
5.5. Gráficas de entrenamiento de PatchTST	58

Índice de tablas

5.1. Tabla de resultados	47
5.2. Tiempos de entrenamiento e inferencia	47
5.3. Días inválidos resultantes tras eliminación de series predichas como inválidas	48
5.4. Días inválidos resultantes tras eliminación de series predichas como inválidas y con umbral optimizado	50
5.5. Errores de FF con umbral 0,5	51
5.6. Errores de FF con umbral 0,25	51
5.7. Errores de CNN con umbral 0,5	52
5.8. Errores de CNN con umbral 0,21	52
5.9. Errores de ViT con umbral 0,5	53
5.10. Errores de ViT con umbral 0,23	53
5.11. Errores de PTST con umbral 0,5	54
5.12. Errores de PTST con umbral 0,18	54

Índice de extractos de código

1.	Obtención de los conjuntos de datos	21
2.	Estratificación de los conjuntos de datos	22
3.	Bucle de entrenamiento	24
4.	Función de entrenamiento	25
5.	Función de evaluación	26
6.	Parámetros para el modelo <i>feedforward</i> (FF)	29
7.	Definición del modelo de red <i>feedforward</i> (FF)	30
8.	Parámetros para el modelo CNN	32
9.	Definición del modelo CNN	33
10.	Parámetros para el modelo ViT	35
11.	Modificación de la definición de ViT para la adaptación al problema	36
12.	Parámetros para el modelo PTST	38
13.	Modificación de la definición de PatchTST para la adaptación al problema	39
14.	Parámetros a optimizar con optimización bayesiana y sus intervalos de búsqueda	40
15.	Código de un modelo CNN parecido al primer prototipo	43

1. Introducción

La radiación es la magnitud física que cualquier instalación solar aprovecha como fuente de energía. A la hora de determinar la viabilidad financiera para cada nuevo proyecto de tecnología solar, es imprescindible tener conocimiento acerca de la calidad y la fiabilidad de esta fuente de energía. No solo eso, esta información es además importante para operaciones de control de precisión, análisis e integración de generación en la red (Sengupta et al. [1]). La variabilidad de la radiación solar representa la mayor fuente de incertidumbre para la predicción del rendimiento de una planta solar. Por lo que la información relativa al recurso solar es necesaria para varias fases de desarrollo y operación de una planta, tales como:

- Selección del lugar de construcción durante estudios de viabilidad.
- Predicción del rendimiento de la planta para su diseño y financiación.
- Medición y predicción a tiempo real para operaciones de la planta y la red.

La selección del lugar para construir una planta de energía solar requiere información acerca de numerosos parámetros, siendo la prioridad más importante si existen recursos solares adecuados. Esto está determinado mayormente por la irradiación anual media, es decir, la cantidad de energía solar por metro cuadrado recibida durante un año de media. Como el tiempo atmosférico puede variar de un año a otro, se considera necesario disponer de un conjunto de datos que aborde los datos de radiación obtenidos durante varios años.

Una vez que se ha elegido un sitio adecuado para la construcción de una planta solar, el desarrollo de un proyecto de energía solar va a requerir de conjuntos de datos más precisos y detallados. Para la optimización de una planta en un lugar específico, la disponibilidad de datos de alta resolución siempre es beneficiosa. Estos datos suelen estar basados en series temporales de radiación solar medidas por estaciones solares cercanas a la planta.

Las estaciones meteorológicas son también valiosas durante la puesta en funcionamiento de grandes plantas. Son esenciales las mediciones solares para estimar el rendimiento a tiempo real para asegurar una alta eficiencia de la planta durante su vida de servicio. De hecho, con el rápido crecimiento de los sistemas

solares con conexión a la red eléctrica, las predicciones de radiación solar han ganado una gran importancia en relación a la seguridad de las operaciones de la red. Las predicciones a corto plazo permiten una producción eficiente, mientras que las predicciones a varios días permiten una planificación eficiente de las reservas.

Las mediciones de radiación solar se obtienen mediante instrumentos sensibles a la radiación, llamados radiómetros. Uno de los más utilizados es el pirheliómetro. Este sensor dispone de un sensor pasivo, o bien fotoeléctrico, o bien termoelectrónico, para convertir la irradiancia solar en una señal eléctrica proporcional. Este sensor mide la irradiancia directa del Sol y, para ello, tiene que seguir su trayectoria a lo largo del día.

Los datos recogidos por estos sensores son series temporales que forman las curvas de radiación solar a lo largo del día. Estas curvas de radiación presentan tanto variabilidad diaria, debido a los cambios de tiempo meteorológico, como estacional. Además, existen varios elementos que pueden dar lugar a errores de medida: suciedad en el sensor, mal alineamiento, problemas eléctricos... todo esto hace que la identificación de errores no sea una tarea trivial, por lo que estas curvas deben ser comprobadas visualmente por expertos.

Hoy en día, con los avances de la Inteligencia Artificial, existen modelos de visión por computador y de predicción de series temporales con capacidad de identificación de patrones que han demostrado una gran tasa de acierto. Es gracias a esto que la tarea de detección de medidas erróneas de radiación se puede automatizar.

En este trabajo se emplearán las mediciones diarias de irradiancia normal directa (DNI) de tres pirheliómetros de una planta de energía termosolar. El fin último de este trabajo es explorar métodos de Aprendizaje Profundo para automatizar el proceso de identificación de irregularidades en las curvas de radiación diarias.

1.1. Fundamentos de radiación solar

La radiación solar es la energía emitida por el Sol en forma de ondas electromagnéticas, que se propaga a través del espacio y llega a la Tierra. Esta es la magnitud física que las instalaciones solares aprovechan como fuente de energía.

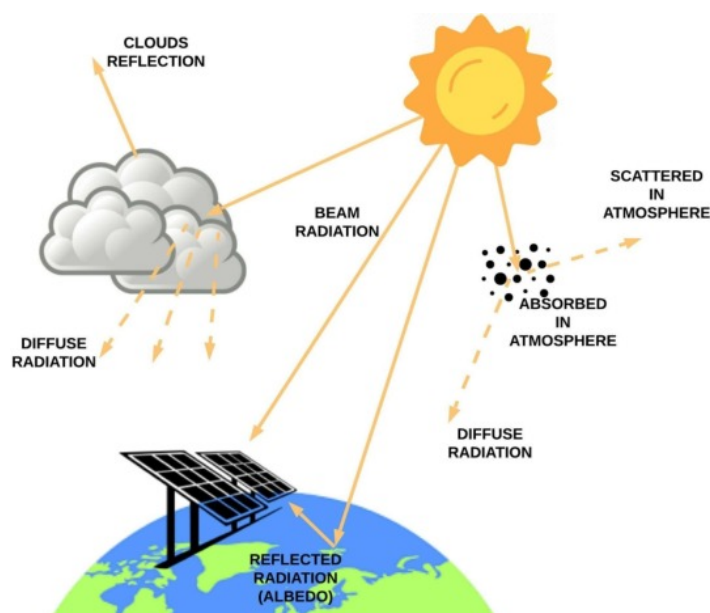


Figura 1.1: Tipos de radiación solar

La radiación solar que se propaga a través de la atmósfera se puede dividir en radiación directa y difusa. La radiación directa es la luz solar que no ha interactuado con los componentes de la atmósfera ni con la superficie terrestre. La radiación difusa es la luz solar dispersada por los componentes atmosféricos o la superficie terrestre (Collins [2]). El esquema de la figura 1.1 muestra estos tipos de radiación solar.

Hay que distinguir la radiación de la irradiancia solar. Esta se define como la potencia incidente (P_{inc}) por unidad de superficie de radiación solar (A_s):

$$I = \frac{P_{inc}}{A_s} \quad (1.1)$$

Existen dos tipos de irradiancia que inciden sobre un lugar concreto de la superficie terrestre:

- La irradiancia directa (Direct Normal Irradiance, DNI). Es la procedente directamente del disco solar.
- La irradiancia difusa (diffuse Horizontal Irradiance, dHI). Es la procedente de toda la bóveda celeste excepto del disco solar.

Cada tipo de planta solar aprovecha un tipo de irradiancia. Las centrales termosolares (figura 1.2) concentran mediante espejos la radiación que proviene directamente del sol en un punto. Esto calienta un fluido, el cual se utiliza para



Figura 1.2: Un tipo de instalación termosolar son los colectores parabólicos, que concentran la radiación en un receptor



Figura 1.3: Placas fotovoltaicas

calentar agua y generar vapor, que finalmente mueve una turbina y produce electricidad. En cambio, las centrales fotovoltaicas (figura 1.3) son capaces de aprovechar todo tipo de radiación, tanto la directa como la difusa.

Un aspecto muy importante a tener en cuenta a la hora de medir la irradiancia es el ángulo de incidencia de los rayos sobre la superficie de medición. El valor de la irradiancia será máximo en una superficie perpendicular al rayo incidente. La DNI se mide de manera perpendicular a los rayos procedentes directamente del disco solar, mientras que la dHI es la irradiancia que incide sobre una superficie horizontal procedente del resto de la bóveda celeste. De aquí la importancia de los distintos ángulos de incidencia de la radiación solar.

Se define la irradiancia directa horizontal (Direct Horizontal Irradiance, DHI) como:

$$DHI = \sin(\alpha)DNI \quad (1.2)$$

Donde α es el ángulo de elevación solar, es decir, el ángulo entre los rayos solares y

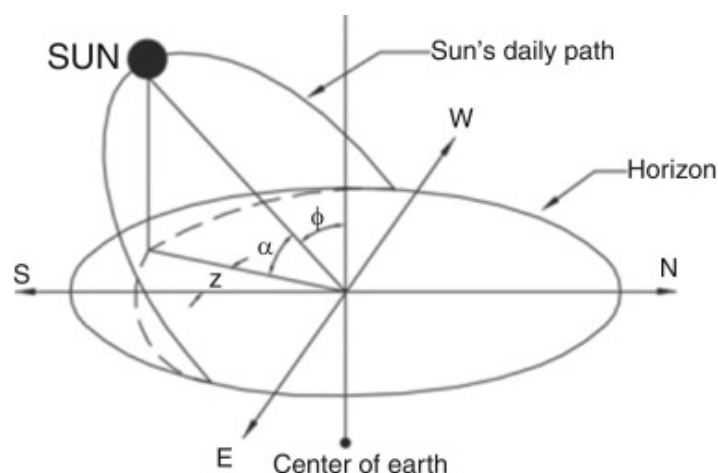


Figura 1.4: Ángulos de elevación solar (α) y cenital (ϕ)

el plano horizontal, tal y como se muestra en la figura 1.4

La DHI representa la radiación que incide horizontalmente sobre la superficie terrestre procedente directamente del disco solar.

Por último, tenemos la irradiancia global (Global Horizontal Irradiance, GHI), que se calcula de la siguiente manera:

$$GHI = dHI + DHI \quad (1.3)$$

Representa toda la irradiancia procedente de la bóveda terrestre.

1.2. Medición de la radiación solar

Disponer de datos fiables relativos al recurso solar es indispensable para el correcto diseño y operaciones de las plantas de energía solar. Eliminando las curvas de radiación inválidas y registrando la media de las curvas de los sensores válidos para cada día del año se consigue minimizar significativamente la influencia de estos errores en el conjunto de datos final.

Los radiómetros son los instrumentos que miden la irradiancia. Esta se mide periódicamente a lo largo del día. Las mediciones se disponen en gráficas donde el eje x es la hora del día y el eje y, la cantidad de irradiancia. De esta manera se forman las llamadas curvas de radiación. En la figura 1.5 se puede ver un ejemplo de curvas de irradiancia directa y de irradiancia global.

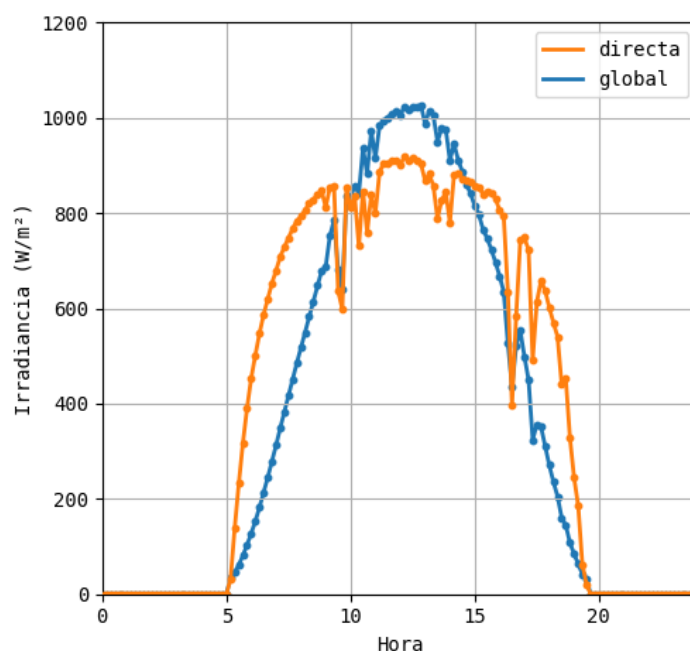


Figura 1.5: Curvas de radiación directa y global

Como se puede ver en la figura, las curvas de radiación forman parábolas a lo largo de las horas del día, con presencia de valles en aquellos instantes en los que haya nubes tapando la luz solar. Durante las horas de noche la irradiancia es nula. Durante la mañana y por la tarde, la irradiancia directa es notablemente mayor a la global, lo cual se debe al bajo ángulo de elevación del Sol a estas horas. A mediodía la irradiancia global puede ser mayor a la directa, esto es gracias al mayor ángulo de elevación y a la suma de la irradiancia difusa procedente del resto de la bóveda celeste.

El diseño, la implementación y la operación de proyectos de plantas de energía solar requieren mediciones precisas de la irradiancia incidente. Dado que las mediciones precisas de irradiancia solar requieren tiempo y un coste económico relativamente altos en comparación con otras variables meteorológicas como la temperatura, estas mediciones solo están disponibles públicamente para un número limitado de estaciones permanentes o semipermanentes. En especial para la DNI o la irradiancia horizontal difusa (DHI) (Sengupta et al. [1]).

Para cada tipo de irradiancia se utilizan diferentes radiómetros especializados. Por un lado, para la DNI se utilizan pirheliómetros (figura 1.6). Estos dispositivos tienen un ángulo de visión reducido, con el que solo pueden capturar la radiación procedente del disco solar. Estos instrumentos van rotando a lo largo del día siguiendo exactamente la trayectoria del Sol en el cielo.



Figura 1.6: Pirheliómetro



Figura 1.7: Pinanómetro

Por otro lado, para la GHI se suelen utilizar pinanómetros (figura 1.7). Este dispositivo está fijo en el suelo y tiene un ángulo de visión mucho más amplio, lo que le permite capturar la radiación proveniente de toda la bóveda celeste.

Las medidas de radiación solar son propicias a errores, es por ello que es necesario ejecutar medidas de control de calidad (QC) para minimizar su influencia en el conjunto de datos obtenido. Una de estas medidas involucra la comparación de las medidas recogidas por sensores redundantes. Emplear tres sensores puede ayudar a determinar cuál presenta errores de medición. En las plantas solares se suelen emplear tres sensores, dispuestos en diagonal, tal y como se indica en la figura 1.8.

Las mediciones recogidas por los sensores redundantes se examinan a diario por expertos para comprobar posibles errores de medición para, de esta manera, descartar las curvas de radiación inválidas y recoger solo los datos de las válidas. En la figura 1.9 se muestra un caso en el que la curva de radiación del sensor S3 no se parece a las de los otros sensores. Esto se debe a errores en la medición. Por tanto, se debe marcar esta curva como inválida para no recopilar estos datos de radiación. En cambio, las medidas tomadas por los tres sensores en el ejemplo de la figura 1.10 sí son válidas, por lo que se pueden utilizar todos estos datos.

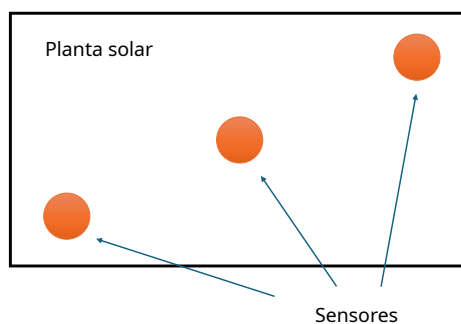


Figura 1.8: Ubicación de los radiómetros dentro de una planta solar

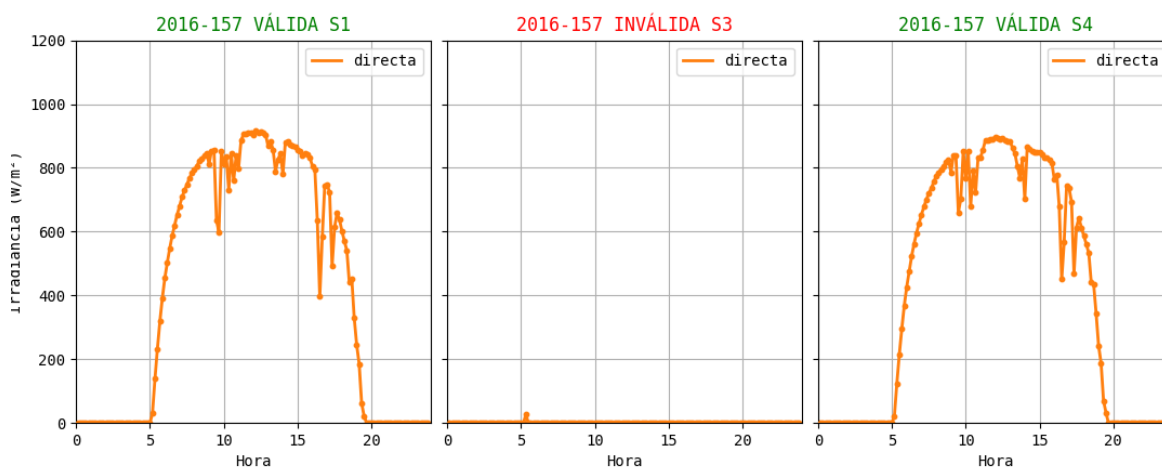


Figura 1.9: Ejemplo de curva de radiación errónea recogida por el sensor S3

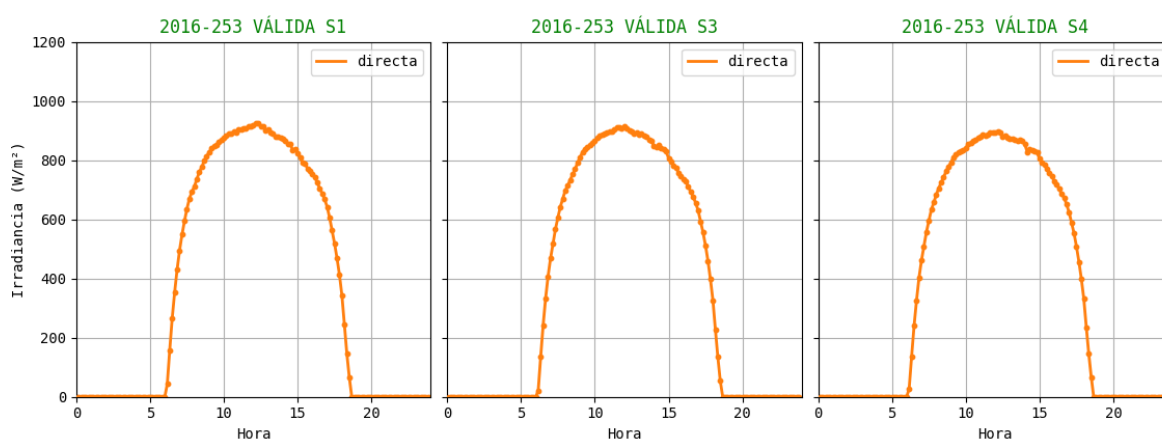


Figura 1.10: Ejemplo de curvas de radiación válidas. No existen mediciones erróneas

Ejemplos de curvas de radiación

Existen muchas causas que pueden causar errores de medidas de DNI, lo que hace que los errores sean diversos entre sí, siendo unos más fáciles de identificar que otros. Además, dado que existe una separación espacial de los sensores, a los tres no les afecta la nubosidad exactamente de la misma manera, con lo que también puede ocurrir que las curvas sean diferentes pero no por ello tengan errores de medición. A continuación se dan algunos ejemplos que permiten ver por qué no es algo trivial la identificación de este tipo de errores.

Para empezar, el fallo encontrado en las curvas de la figura 1.9 es muy evidente, pero existen muchos casos, como en el de la figura 1.11 donde no lo es tanto.

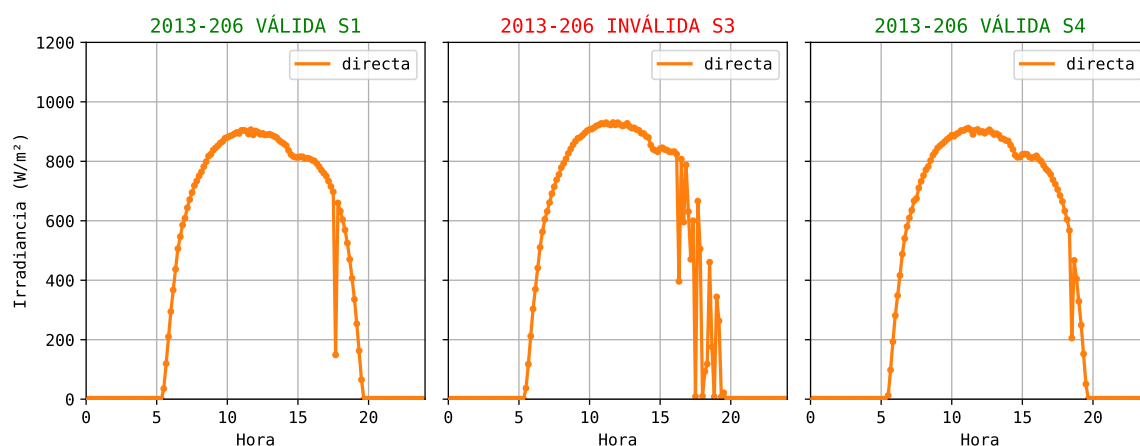


Figura 1.11: Ejemplo de error poco evidente

En la figura 1.12 se puede ver un ejemplo de fallo por mal alineamiento. La curva inválida está desplazada a partir de la hora 15. Este fallo es mucho más evidente en la figura 1.13.

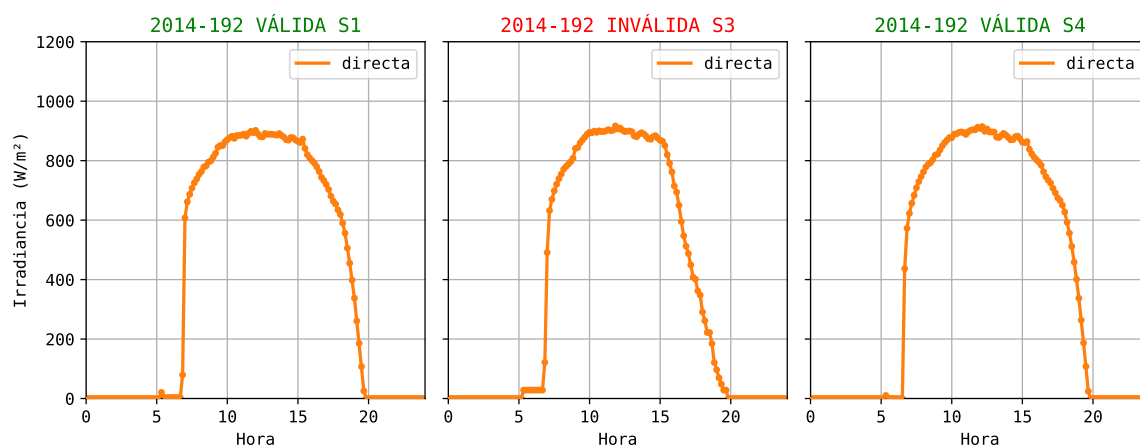


Figura 1.12: Ejemplo 1 de error de mal alineamiento

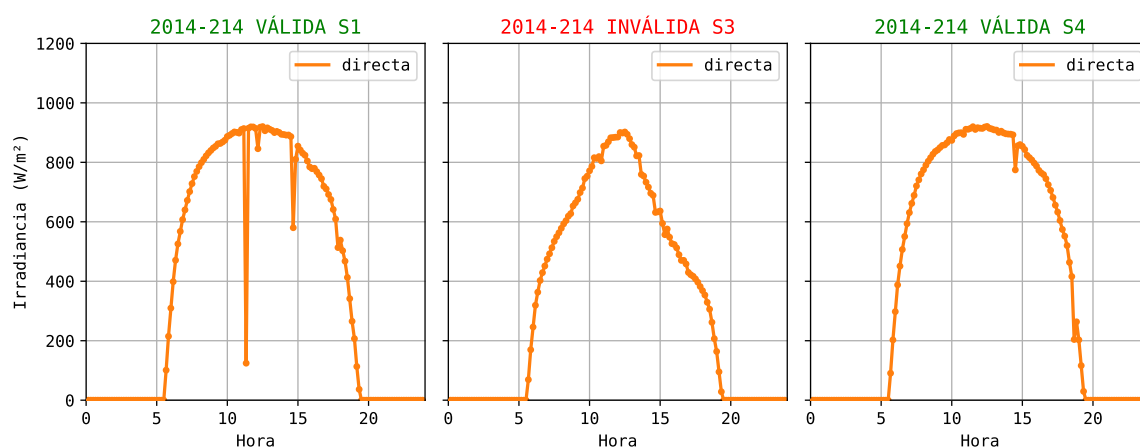


Figura 1.13: Ejemplo 2 de error de mal alineamiento

Un aspecto importante a destacar es que existen valores perdidos en las curvas. Este tipo de error se debe a fallos eléctricos del sensor. Para imputar los valores faltantes, ya que representan también errores de medición, lo que se hará es asignarles un valor de irradiancia anormalmente elevado. Un ejemplo de esto se muestra en la figura 1.14, donde los valores faltantes se encuentran alrededor de las horas 18 y 20. Existen otros casos donde este error se da a lo largo de todas las horas de luz, como es el caso de la figura 1.15

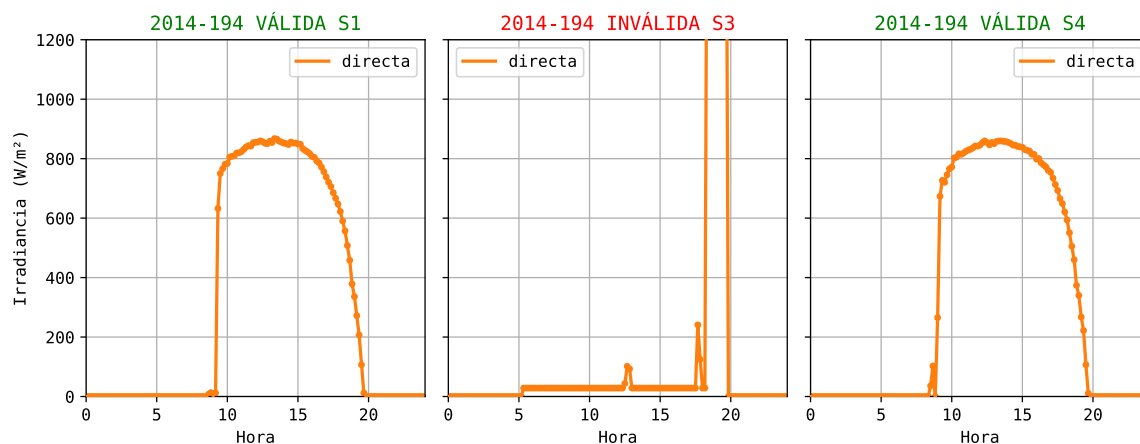


Figura 1.14: Ejemplo 1 de error con valores perdidos

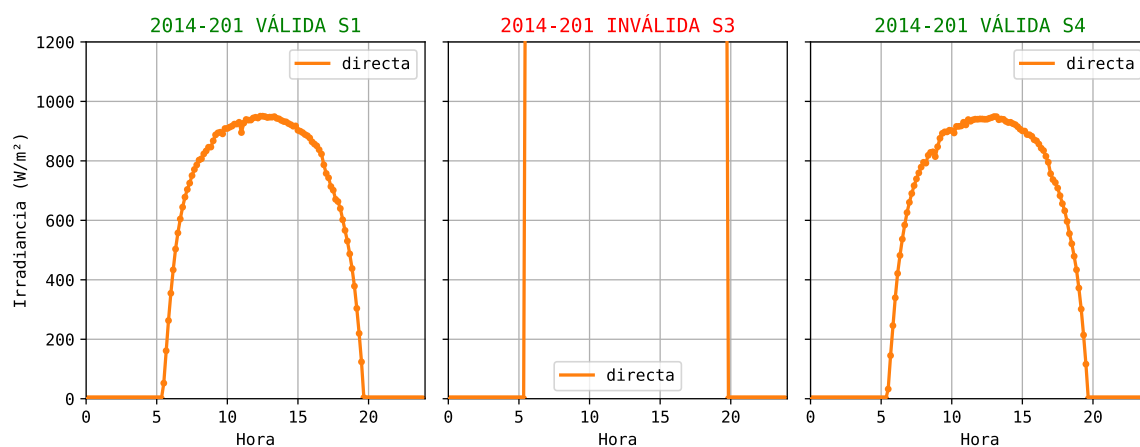


Figura 1.15: Ejemplo 2 de error con valores perdidos

Un ejemplo en el que existen dos curvas inválidas es el de la figura 1.16.

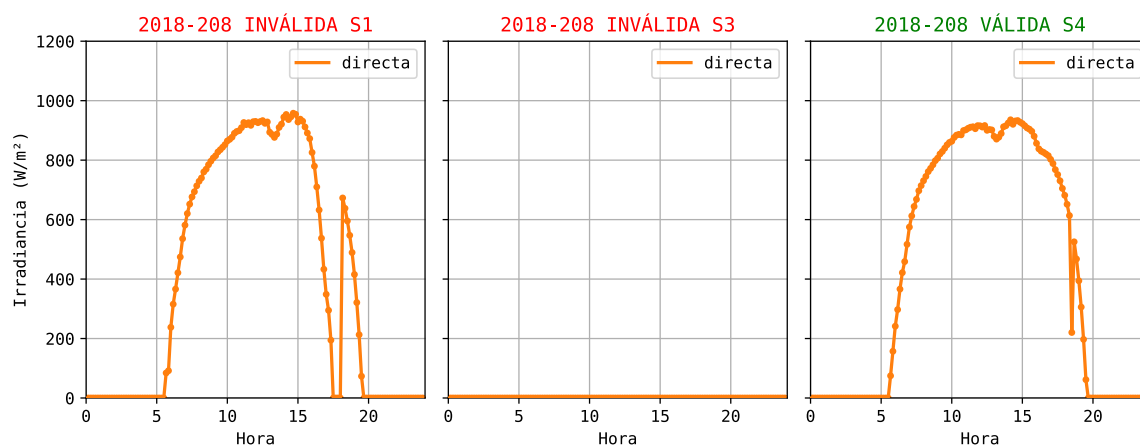


Figura 1.16: Ejemplo de error múltiple

Otro tipo de fallo común es la suciedad presente en el sensor. Esto hace que la curva sea más baja respecto a las otras, tal y como se ve en el ejemplo de la figura 1.17. Este tipo de fallo puede ser difícil de identificar si la diferencia es pequeña.

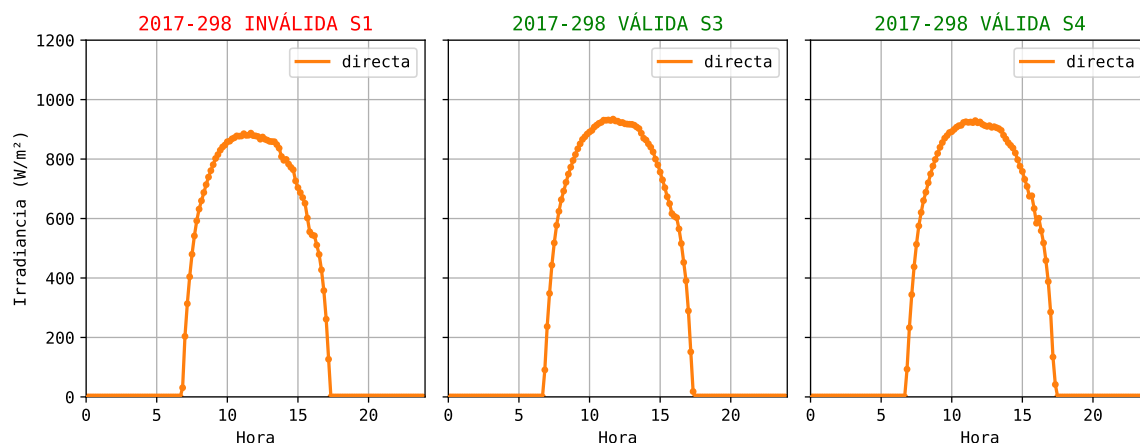


Figura 1.17: Ejemplo de error por suciedad

En ocasiones, los fallos pueden pasar desapercibidos al confundirse por los valles en la curva que provoca la presencia de nubes. Es el caso del fallo en la figura 1.18.

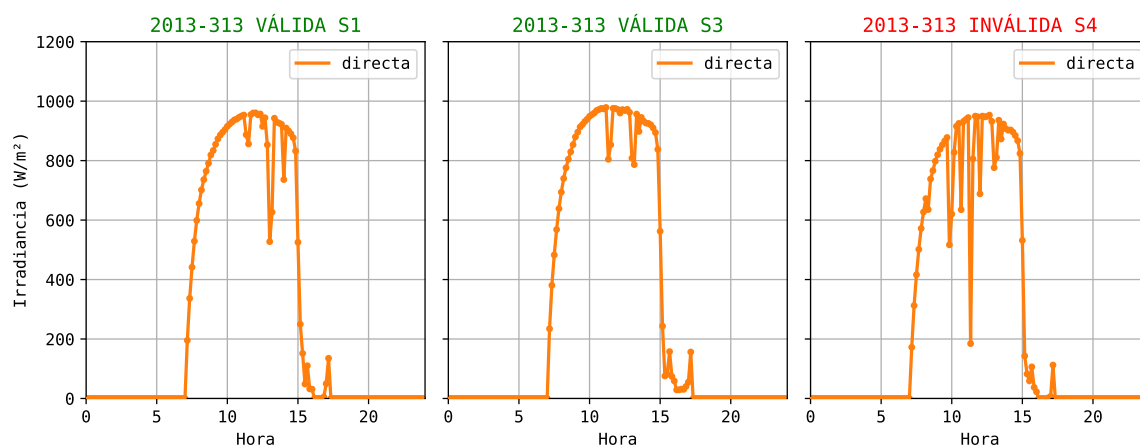


Figura 1.18: Ejemplo de error con nubosidad

Hasta ahora, exceptuando el caso de los valores perdidos, los fallos de medición siempre han sido valores inferiores a los proporcionados por las curvas válidas, lo que quiere decir que el sensor en un momento determinado no ha podido capturar toda la irradiancia procedente del Sol. Sin embargo, también existen casos en los que la curva inválida está por encima de lo normal, como es el caso de la figura 1.19, cerca de la hora 5.

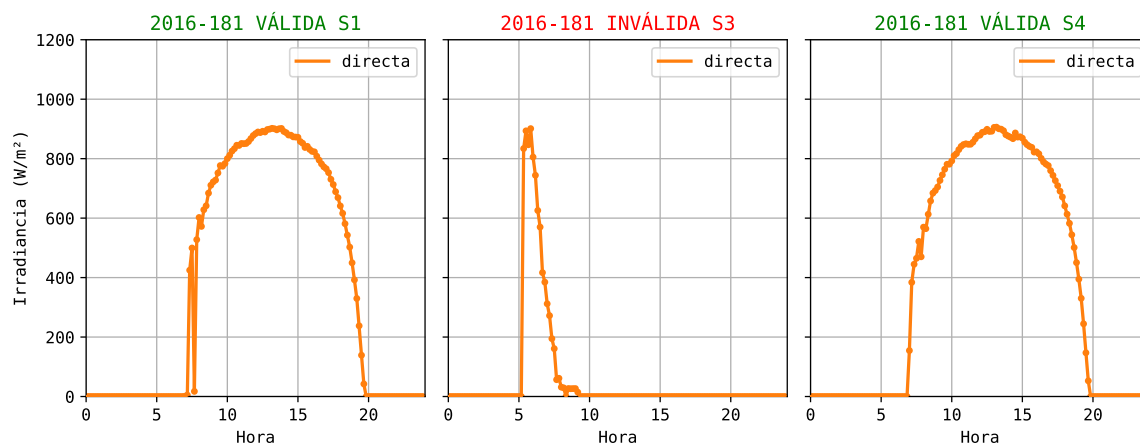


Figura 1.19: Ejemplo de error anómalo

También existen casos en los que el fallo es dudoso, como en el de la figura 1.19. Aquí, a las primeras horas del día la curva del sensor inválido es inferior. El problema es que el día es nublado, con lo que la diferencia puede ser debida a mayor presencia de nubosidad en ese momento para la ubicación de ese sensor.

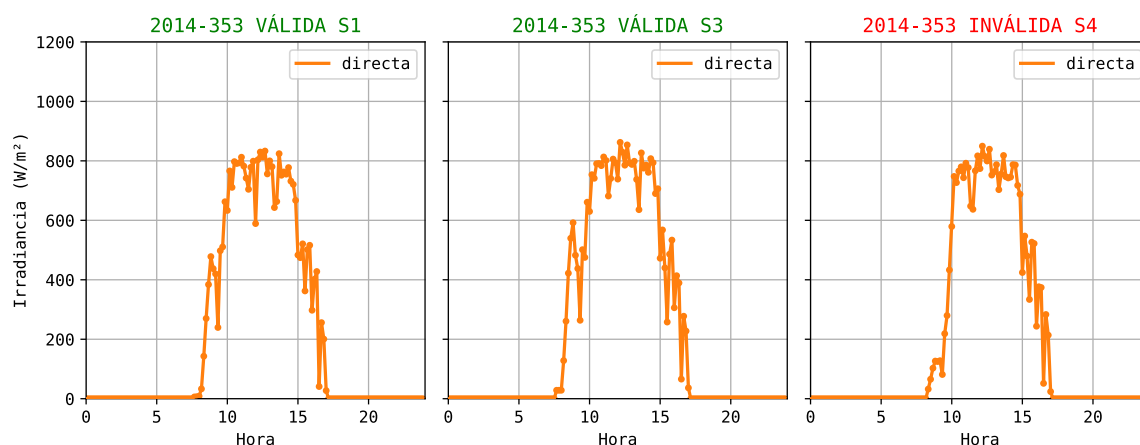


Figura 1.20: Ejemplo de error dudoso

Como ya se ha mencionado, la nubosidad provoca diferencias a lo largo de los instantes de medición entre las curvas de cada sensor. En días donde predominan los intervalos nublados tales como los de la figura 1.21 es más probable que el modelo de aprendizaje automático detecte estas diferencias y marque como inválido un día que no lo es.

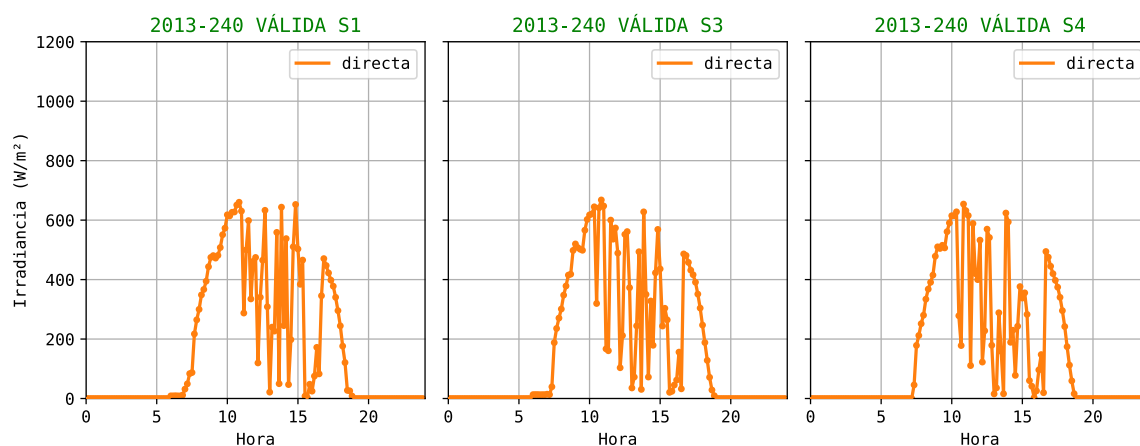


Figura 1.21: Ejemplo de series válidas con día nublado

Por tanto, es difícil que solamente con medidas estadísticas se pueda determinar de forma automática si una curva de radiación presenta errores de medición. Los modelos de aprendizaje profundo más avanzados tienen la habilidad de identificar los patrones y matices que distinguen a unos casos de otros. Con lo que pueden ser un recurso útil para la identificación de los errores de medición de radiación solar.

1.3. Avances de la IA en detección de errores

En los últimos años, el aprendizaje profundo ha transformado el campo de la detección de errores y anomalías en datos de sensores, gracias a su capacidad para aprender representaciones complejas y patrones sutiles en grandes volúmenes de datos. Entre las principales técnicas utilizadas destacan las redes neuronales convolucionales (CNN) y los modelos basados en transformers, los cuales han mostrado un rendimiento sobresaliente en tareas de clasificación, detección de patrones y predicción en series temporales.

Redes Neuronales Convolucionales (CNN)

Las CNN, originalmente diseñadas para el procesamiento de imágenes, han demostrado también gran eficacia en el análisis de series temporales y datos sensoriales. Su capacidad para extraer automáticamente características locales relevantes mediante filtros convolucionales las convierte en una herramienta potente para detectar patrones anómalos o errores en las mediciones. Modelos CNN en combinación con módulos RNN o LSTM para capturar patrones espaciales y temporales se han utilizado con éxito para la detección de anomalías

en series temporales. Tales son los ejemplos de Canizo et al. [3] y Liu et al. [4].

Transformers y su impacto en la detección de errores

En paralelo, los modelos transformer han revolucionado el análisis de secuencias. Su mecanismo de atención, capaz de capturar relaciones de largo alcance en los datos, los ha hecho especialmente adecuados para tareas de clasificación y predicción en series temporales, superando en muchos casos a las arquitecturas recurrentes tradicionales.

Uno de los hitos clave fue la introducción del Vision Transformer (ViT) por Dosovitskiy et al. [5], en su artículo "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale". Este modelo trasladó la estructura del transformer al dominio de visión por computadora, dividiendo las imágenes en parches (patches) y tratándolos como tokens, similar al procesamiento de texto. Esta idea permitió que los transformers superaran a las CNN en tareas de clasificación de imágenes a gran escala.

Inspirados por este avance, otros investigadores adaptaron el enfoque a las series temporales. Un ejemplo destacado es el modelo PatchTST, presentado por Nie et al. [6] en su trabajo "A Time Series is Worth 64 Words: Long-term Forecasting with Transformers". PatchTST aplica la idea de dividir series temporales en patches, análogos a ViT, para mejorar el rendimiento en tareas de predicción. Esta técnica ha logrado resultados de vanguardia en benchmarks de predicción de largo plazo así como en aprendizaje de la representación, abriendo nuevas posibilidades para la detección de errores en datos de sensores. Por ejemplo, el artículo Li et al. [7] introduce un modelo basado en PatchTST para la detección de anomalías en series temporales de electrocardiogramas (ECG).

Relevancia para la detección de errores en sensores de radiación solar

La detección de errores en mediciones de radiación solar presenta desafíos específicos debido a la diversidad de tipos de errores y a la variabilidad temporal y espacial de las mediciones. Los modelos antes mencionados ofrecen enfoques prometedores:

- Las CNN pueden capturar patrones locales y transitorios asociados a errores abruptos o picos anómalos.
- Los transformers son adecuados para modelar dependencias de largo alcance y patrones recurrentes, facilitando la identificación de anomalías sutiles o

errores persistentes.

Por tanto, la aplicación de estas arquitecturas avanzadas permite diseñar sistemas robustos y precisos para la clasificación y detección automática de errores en mediciones de radiación de sensores solares.

1.4. Objetivos

El objetivo principal de este proyecto es desarrollar un sistema automatizado y sistemático para la detección de errores en la medición de la radiación directa normal (DNI) en plantas termosolares. Para lograrlo, se implementarán diversos modelos de inteligencia artificial con el fin de optimizar y automatizar este proceso. La meta final es integrar esta solución en el sistema de supervisión operativa de una central termosolar, mejorando así la fiabilidad y eficiencia del monitoreo en tiempo real.

1.5. Estructura del documento

Una vez conocido el marco teórico de este trabajo, en el capítulo 2 se detallará el conjunto de datos utilizado así como el flujo de trabajo seguido a lo largo de toda la elaboración del trabajo. Aparte, en el capítulo 3 se detallará el diseño concreto de los modelos de aprendizaje profundo utilizados y adaptados a esta tarea.

Seguidamente, se dedicará el capítulo 4 a explicar las principales dificultades que han debido de ser abordadas a lo largo del trabajo requiriendo un esfuerzo y dedicación adicionales, así como las soluciones que se les han dado.

En el capítulo 5 se mostrarán los resultados de los modelos, mostrando las métricas principales de clasificación y el rendimiento de cada uno de los modelos. Posteriormente, en el capítulo 6 se hará un análisis crítico acerca de los resultados obtenidos y sus relaciones con las características de los modelos.

Por último, en el capítulo 7 se concluirá este documento, recopilando el trabajo realizado y los resultados obtenidos. También se propondrán mejoras y posibles líneas de investigación para trabajos futuros.

2. Metodología

2.1. Herramientas utilizadas

Para el desarrollo de este trabajo se ha empleado el lenguaje de programación Python, junto con varias bibliotecas especializadas en computación científica y aprendizaje automático, entre las que destacan Pytorch, Matplotlib y Scikit-learn. A continuación, se justifica su elección:

Python ha sido elegido por ser uno de los lenguajes más utilizados en el ámbito del aprendizaje automático y la ciencia de datos. Sus principales ventajas son:

- Sintaxis clara y legible, lo que facilita la escritura y mantenimiento del código.
- Amplia comunidad y gran cantidad de recursos disponibles, incluyendo documentación, tutoriales y foros de soporte.
- Riqueza del ecosistema de librerías científicas, lo que permite cubrir todo el ciclo de desarrollo de modelos de aprendizaje automático: desde el preprocesamiento de datos hasta la visualización y entrenamiento de modelos.

Matplotlib se ha empleado como herramienta principal de visualización de datos. Sus ventajas son:

- Posibilita la creación de gráficos estáticos de alta calidad (líneas, histogramas, imágenes, etc.), lo cual es crucial para el análisis exploratorio y la interpretación de resultados.
- Es altamente personalizable y se integra bien con otras librerías como NumPy y Pandas.

PyTorch ha sido la librería seleccionada para implementar los modelos de aprendizaje profundo, principalmente por:

- Su flexibilidad y dinamismo, gracias a un modelo de grafos computacionales dinámicos (*define-by-run*), lo que facilita la depuración y experimentación.

- Ofrece una amplia variedad de herramientas para el desarrollo de redes neuronales, incluyendo optimizadores, funciones de pérdida, y módulos predefinidos.
- Cuenta con una comunidad muy activa en el ámbito académico y profesional, lo que la convierte en una de las bibliotecas más relevantes en la actualidad para investigación y producción.

Scikit-learn es una de las bibliotecas más completas y consolidadas para el desarrollo de algoritmos de aprendizaje automático clásicos en Python. Se ha empleado para la división del conjunto de datos en entrenamiento, validación y prueba de forma estratificada.

Einops es una biblioteca que permite realizar operaciones sobre tensores de forma flexible y que produce un código legible y fiable (Rogozhnikov [8]). Esta biblioteca ha sido empleada para modificar la estructura de los tensores de datos para los algoritmos que lo requieren.

BayesianOptimization (Nogueira [9]) es una biblioteca que permite la implementación de optimización global bayesiana mediante procesos gaussianos. Esta librería ha sido utilizada para optimizar los hiperparámetros de los modelos de aprendizaje profundo. Esta estrategia se utiliza para balancear la exploración y la explotación en la búsqueda de hiperparámetros, consiguiendo buenos resultados minimizando la cantidad de entrenamientos necesarios. Los parámetros son tanto continuos como discretos, para estos últimos se emplea el algoritmo presentado por Garrido-Merchán and Hernández-Lobato [10].

vit-pytorch (lucidrains [11]) es una biblioteca que incluye implementaciones en Pytorch del algoritmo original de transformer visual presentado en Dosovitskiy et al. [5] así como de modificaciones del mismo. Para este trabajo se ha utilizado la implementación del algoritmo original.

Transformers es una biblioteca creada por la comunidad de Hugging Face que contiene una recopilación de los modelos de transformers más notables creados hasta el momento. Esta recopilación viene presentada en el artículo Wolf et al. [12]. En este trabajo se ha utilizado esta librería para crear el modelo transformer PatchTST, presentado en el trabajo Nie et al. [6]. Este modelo está pensado para la predicción de series temporales multivariadas. Se ha decidido utilizar esta biblioteca puesto que ya consta con una versión de este modelo transformer preparada para la clasificación de series temporales.

2.2. Conjunto de datos

Se han utilizado los datos relativos a las mediciones de irradiancia directa (DNI) de una central solar. Los sensores de la central a partir de los que se obtienen las medidas de DNI son los número 1, 3 y 4. De aquí los nombres de cada sensor: S1, S3 y S4. Los datos se componen de mediciones de estos tres sensores durante los años 2013 a 2018 y la primera mitad del 2019, lo que equivale a 2371 días. Las mediciones se realizan en intervalos de 10 minutos. De estos datos se han extraído series temporales diarias, las cuales constan de 24 horas * 6 mediciones/hora = 144 mediciones. Para cada día hay una serie temporal por cada sensor.

En resumen, el conjunto de datos consta de:

- 144 mediciones por serie temporal
- 3 series temporales por día (una por sensor)
- 2371 días

Los datos se han obtenido de una tabla Excel (figura 2.1). En esta tabla vienen marcados en rojo los días inválidos de cada sensor. Para poder manejarlos fácilmente, el conjunto de datos se ha estructurado de la manera que indica la figura 2.2: para cada sensor existe un directorio para las series válidas y otro para las no válidas. En estos directorios se almacenan los ficheros nombrados con el año y el día de la serie temporal que almacenan.

	A	B	C	D	E	F	G	H	I	J	K
1	Año	Mes	Día	Hora	Minuto	DNI_S1	DNI_S3	DNI_S4	GHI_S1	GHI_S3	GHI_S4
528	2013	1	19	5	40	0	0	0	0	0	0
529	2013	1	19	5	50	0	0	0	0	0	0
530	2013	1	19	6	0	0	0	0	0	0	0
531	2013	1	19	6	10	0	0	0	0	0	0
532	2013	1	19	6	20	0	0	0	0	0	0
533	2013	1	19	6	30	0	0	0	0	0	0
534	2013	1	19	6	40	0	0	0	0	0	0
535	2013	1	19	6	50	0	0	0	0	0	0
536	2013	1	19	7	0	0	0	0	0	0	0
537	2013	1	19	7	10	0	0	0	0	0	0
538	2013	1	19	7	20	0	0	0	0	0	0
539	2013	1	19	7	30	0	0	0	0	0	0
540	2013	1	19	7	40	7,07781124	3,4000001	408,099457	4,75577211	2,93302298	360,3992
541	2013	1	19	7	50	7,06161642	3,4000001	169,571838	3,47630787	2	388,244965
542	2013	1	19	8	0	6,96896076	3,4000001	318,681366	5,98814201	3,39697313	318,681366
543	2013	1	19	8	10	6,39756012	3,4000001	363,876282	11,9857769	11,4828157	379,253815
544	2013	1	19	8	20	7,27872324	3,47748375	225,752289	9,57826996	9,77607059	403,966034
545	2013	1	19	8	30	7,56450748	5,6916995	414,293793	12,1211529	12,842247	372,138855
546	2013	1	19	8	40	8,29363918	3,4000001	353,483612	25,6311378	23,8087273	342,91272

Figura 2.1: Tabla Excel con los datos de irradiancia

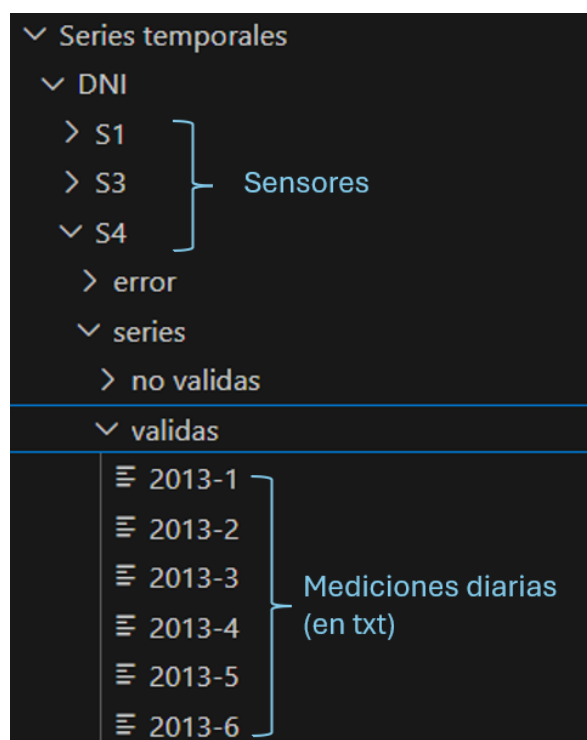


Figura 2.2: Árbol de directorios en el que se estructura el conjunto de datos

Seguidamente, se ha creado un programa para visualizar las curvas de radiación y su etiqueta, tal y como se muestra en las figuras 1.9 y 1.10. Con este programa se han revisado visualmente las series temporales para buscar posibles errores de clasificación. Finalmente, se ha modificado el conjunto de datos original, cambiando las etiquetas de las series temporales incorrectamente clasificadas.

2.3. Implementación

En esta sección se expondrá la implementación que se sigue en el programa principal, desde la carga del conjunto de datos hasta la obtención de los resultados finales. Ha habido que hacer más código que el aquí mostrado. Toda la implementación realizada se encuentra en el repositorio <https://github.com/amogue73/TFM-MULCIA>

En primer lugar, se ha cargado el conjunto de datos ya corregido. La estructura de datos que se ha elegido para representarlo es la siguiente:

- Como conjunto de atributos se han tomado las series temporales. Cada instancia consiste en las tres series temporales diarias de sendos sensores.

- Como conjunto de etiquetas se ha tomado para cada instancia tres valores booleanos que representa la validez de cada serie temporal: 0 si es válida y 1 si es inválida.

Seguidamente, se ha dividido el conjunto de datos en conjuntos de entrenamiento, validación y prueba. La selección de instancias para cada conjunto se ha realizado de manera aleatoria, pudiendo tener cada conjunto instancias de cualquier día. El porcentaje que compone cada conjunto de datos es el siguiente:

- Entrenamiento: 60 %
- Validación: 20 %
- Prueba: 20 %

La división se ha hecho de manera estratificada. Como para cada instancia existen tres valores objetivos, no hay una manera directa de estratificar los datos. Para afrontar esta dificultad se ha decidido crear un atributo auxiliar que represente cada posible combinación de valores booleanos para las tres series temporales. Esto es, la combinación (0, 0, 0) recibe el valor 0, la combinación (0, 0, 1) recibe el valor 1, la combinación (0, 1, 0) recibe el valor 2, etcétera. De esta manera, en total se obtienen 8 categorías para este nuevo atributo, que sirve para estratificar la división del conjunto de datos.

```
batch_size = 16
importlib.reload(get_dataset)
train_ds, test_ds, val_ds, weights = get_dataset.get_dataset(\
0.20,0.20,sequential=False,coef_weights=1,random_state=42,device=device)

train_loader = DataLoader(train_ds,batch_size=batch_size,shuffle=False,drop_last=True)
test_loader = DataLoader(test_ds,batch_size=batch_size,shuffle=False,drop_last=True)
val_loader = DataLoader(val_ds,batch_size=batch_size,shuffle=False,drop_last=True)
```

Extracto de código 1: Obtención de los conjuntos de datos

```
def label_to_categories(label):
    return int(''.join(map(str, [int(label[i]) for i in range(len(label))])), 2)

def get_dataset(test_prop, val_prop, sequential, coef_weights=1, random_state=None, device="cpu"):

    series = reshape_series(get_series())
    val_relative_size = val_prop / (test_prop + val_prop)

    if not sequential:
        train_series, temp = train_test_split(
            series,
            test_size=(test_prop + val_prop),
            random_state=random_state,
            stratify=[label_to_categories(series[i][1]) for i in range(len(series))]
        )

        val_series, test_series = train_test_split(
            temp,
            test_size=(1 - val_relative_size),
            random_state=random_state,
            stratify=[label_to_categories(temp[i][1]) for i in range(len(temp))]
        )
    else:
        train_series = series[:round((1-test_prop)*len(series))]
        test_series = series[round((1-test_prop)*len(series)):]
        val_series = None

    train_ds = dni(PATH_D, train_series, device)
    test_ds = dni(PATH_D, test_series, device)
    val_ds = dni(PATH_D, val_series, device)
    weights = get_weights(train_series, coef_weights)

    cat_train = np.array([label_to_categories(train_series[i][1])\
        for i in range(len(train_series))])
    cat_val = np.array([label_to_categories(val_series[i][1])\
        for i in range(len(val_series))])
    cat_test = np.array([label_to_categories(test_series[i][1])\
        for i in range(len(test_series))])

    return train_ds, test_ds, val_ds, weights
```

Extracto de código 2: Estratificación de los conjuntos de datos

En relación a los conjuntos de datos, cabe decir que el tamaño de batch seleccionado es de 16. Este tamaño ofrece una buena proporción respecto a este conjunto de datos, evitando ruido y dando lugar a un aprendizaje suave y eficiente.

Antes de empezar el entrenamiento, con el fin de obtener el mejor rendimiento posible, se ha decidido hacer una búsqueda de los hiperparámetros que optimicen cada modelo. Para ello se ha seguido la estrategia de optimización bayesiana. Esta técnica permite hacer una búsqueda que maximice el resultado obtenido sin necesidad de realizar un gran número de entrenamientos y evaluaciones. Para cada algoritmo, en total se han realizado 50 entrenamientos y evaluaciones con diferentes combinaciones de hiperparámetros. Para esta tarea han sido utilizados solamente los conjuntos de entrenamiento y de validación; el conjunto de prueba queda reservado para la evaluación final.

Este paso ha sido de gran ayuda para decidir los hiperparámetros de los modelos finales con los que realizar la experimentación. Unos hiperparámetros sometidos al proceso de búsqueda han sido la tasa de aprendizaje y el umbral para la clasificación. Sin embargo, finalmente se ha optado por darles el mismo valor de estos hiperparámetros a todos los modelos de aprendizaje. Esta decisión se ha tomado porque esto permite comparar de manera más justa elementos tales como el balance precisión-sensibilidad o el ritmo de aprendizaje.

Una vez que se han definido los hiperparámetros de los modelos, el siguiente paso es proceder con la experimentación. Los modelos se han entrenado con el conjunto de entrenamiento, mientras que el conjunto de validación ha sido utilizado para determinar la mejor época del entrenamiento y devolver los parámetros del modelo de esta época. Cada modelo se ha entrenado siguiendo los siguientes criterios:

- Se ha establecido el número máximo de épocas en 60. Una vez realizada la época 60, el modelo para de entrenar.
- Se ha elegido un valor de paciencia de 20. Si pasadas 20 épocas no se ha conseguido mejorar el modelo, se interrumpe el aprendizaje del modelo. De esta manera se evita un posible sobreajuste.
- Se elige como mejores parámetros del modelo aquellos que obtengan el mejor valor F1 con la condición adicional de que la sensibilidad sea superior al 80 %, con la excepción del modelo base, *feedforward*, que en este caso es del 70 %.

Una vez que se ha obtenido los mejores parámetros para cada modelo, estos se guardan y se utilizan para clasificar el conjunto de prueba. Los valores de rendimiento obtenidos de esta clasificación son los que se describirán en las tablas de rendimiento en el capítulo de experimentación.

```

print("MODEL: " + model.name)
for t in range(epochs):
    print(f"Epoch {t+1}\n-----")
    tiempo_entrenamiento += train_loop(train_loader, model, criterion, optimizer)
    print("Training dataset:")
    loss[0,t], accuracy[0,t], precission[0,t], recall[0,t], f1[0,t], tiempo = \
    test_loop(train_loader, model, criterion, threshold=thres, verbose=True)
    tiempo_inferencia += tiempo
    print("")
    print("Validation dataset:")
    loss[1,t], accuracy[1,t], precission[1,t], recall[1,t], f1[1,t], tiempo = \
    test_loop(val_loader, model, criterion, threshold=thres, verbose=True)
    tiempo_inferencia += tiempo
    print("")

    if (f1[1,t] > best_score):
        epochs_sin_mejora = 0
        if (recall[1,t] > min_recall):
            best_score = f1[1,t]
            best_epoch = t+1
            torch.save(model.state_dict(), 'mejor_modelo_' + model.name + name_add + '.pth')
    else:
        epochs_sin_mejora += 1

    if epochs_sin_mejora == patience:
        print("PARADA TEMPRANA")
        break

print("Mejor valor de F1 encontrado para " + model.name + \
f": {best_score} (epoch: {best_epoch})")
print("")
best_f1_list.append(best_score)
best_epoch_list.append(best_epoch)
model.load_state_dict(torch.load('mejor_modelo_' + model.name + name_add + '.pth'))
test_loop(test_loader, model, criterion, thres, True)
print("Tiempo de entrenamiento:", tiempo_entrenamiento)
print("Tiempo de inferencia: ", tiempo_inferencia)
show_metrics(t+1, loss[:, :t+1], accuracy[:, :t+1], precission[:, :t+1], \
recall[:, :t+1], f1[:, :t+1], best_epoch, model.name + name_add)

```

Extracto de código 3: Bucle de entrenamiento

```
def train_loop(dataloader, model, loss_fn, optimizer):

    t1 = time()
    model.train()
    patchtst = model.name == "PatchTST"

    for batch, (X, y) in enumerate(dataloader):
        if patchtst:
            X = rearrange(X, 'b c s -> b s c')

            # Compute prediction and loss
            pred = model(X)
            loss = loss_fn(pred, y)

            # Backpropagation
            loss.backward()
            optimizer.step()
            optimizer.zero_grad()

    t2 = time()
    return t2 - t1
```

Extracto de código 4: Función de entrenamiento

```
def test_loop(dataloader, model, loss_fn, threshold, verbose=True):
    t1 = time()
    model.eval()
    patchtst = model.name == "PatchTST"
    num_batches = len(dataloader)
    positive_real, positive_pred, test_loss, correct, correct_pos = 0, 0, 0, 0, 0
    with torch.no_grad():
        for X, y in dataloader:
            if patchtst:
                X = rearrange(X, 'b c s -> b s c')
            pred = model(X)
            loss = loss_fn(pred, y).item()
            test_loss += loss
            pred_pos = torch.where(F.sigmoid(pred) > threshold, 1, 0)
            real_pos = (y == 1.0)
            positive_pred += pred_pos.type(torch.float).sum().item()
            positive_real += y.type(torch.float).sum().item()
            correct_pos += torch.bitwise_and(pred_pos, real_pos).type(torch.float).sum().item()
            correct += (pred_pos == y).type(torch.float).sum().item()

    test_loss /= num_batches
    correct /= 3*num_batches*dataloader.batch_size
    if (positive_pred != 0):
        precission = correct_pos/positive_pred
    else:
        precission = 0.0
    if (positive_real != 0):
        recall = correct_pos/positive_real
    else:
        recall = 0.0
    if (precission+recall > 0):
        f1score = 2*precission*recall/(precission+recall)
    else:
        f1score = 0
    if (verbose):
        print(f"Accuracy: \t {(100*correct):>0.1f}%, Avg loss: {test_loss:>8f}")
        print(f"Precision: \t {(100*precission):>0.1f}%")
        print(f"Recall: \t {(100*recall):>0.1f}%")
        print(f"F1-score: \t {(f1score):>0.3f}")
    t2 = time()
    return test_loss, correct, precission, recall, f1score, t2-t1
```

Extracto de código 5: Función de evaluación

3. Diseño de modelos

Este capítulo está dedicado al diseño de los modelos de aprendizaje profundo que se han utilizado en la experimentación. Los modelos son generalizables a cualquiera cantidades de sensores y de mediciones. Por tanto, se determinan las siguientes variables:

- N_S es el número de sensores que realizan mediciones de DNI.
- N_M es el número de medidas que cada sensor realiza a lo largo de un día.

En este caso se tiene que $N_S = 3$ y $N_M = 144$.

Los modelos que se han diseñado son los siguientes:

1. Red *feedforward* (FF) simple de dos capas ocultas. Este modelo será el *baseline* que servirá para determinar la mejoría que modelos más complejos aportan.
2. Red neuronal convolucional (CNN). Este modelo se basa en las redes convolucionales clásicas para clasificación de imágenes. Para ello, las series temporales se convierten convenientemente en una matriz, que formaría la imagen que el modelo adquiere como entrada.
3. Transformer de visión (ViT). Siguiendo con la misma estrategia, se crea una matriz que forma la imagen de entrada de este modelo. Pero en este caso, se utiliza un modelo transformer.
4. PatchTST (PTST). Este es también un modelo transformer. A diferencia del anterior, no está preparado para trabajar con imágenes sino con series temporales.

Un elemento que todos los modelos tienen en común es la arquitectura que genera las salidas. La última capa del modelo tiene tantas neuronas como número de sensores (N_S). Ellas se pasan a una función logística para transformar su salida en el intervalo $(0, 1)$.

La función de coste es la entropía cruzada binaria (BCE), que toma las salidas de esta función junto con los valores de las etiquetas:

$$L = \sum_i^{N_S} BCE(\sigma(o_i), y_i) \quad (3.1)$$

Donde i determina el número de sensor, o_i es la salida de la última capa de la red neuronal, y_i es la etiqueta, ambos correspondiente al sensor i y σ es una función sigmoide.

Para la clasificación, una serie temporal se clasifica como 1 (inválida) si la salida de la función logística supera un determinado umbral, y se clasifica como 0 (válida) en caso contrario:

$$\hat{y}_i = \begin{cases} 1 : \sigma(o_i) > \psi \\ 0 : \sigma(o_i) \leq \psi \end{cases} \quad (3.2)$$

Donde $\psi \in (0, 1)$ es el umbral.

A continuación, para cada modelo se mostrará los hiperparámetros que se han utilizado. Para FF y CNN también se mostrará sus definiciones en Pytorch, mientras que para ViT y PTST se mostrará la modificación del código original [11, 13] que se han realizado para adaptarlos a este problema.

3.1. Red *feedforward* (FF)

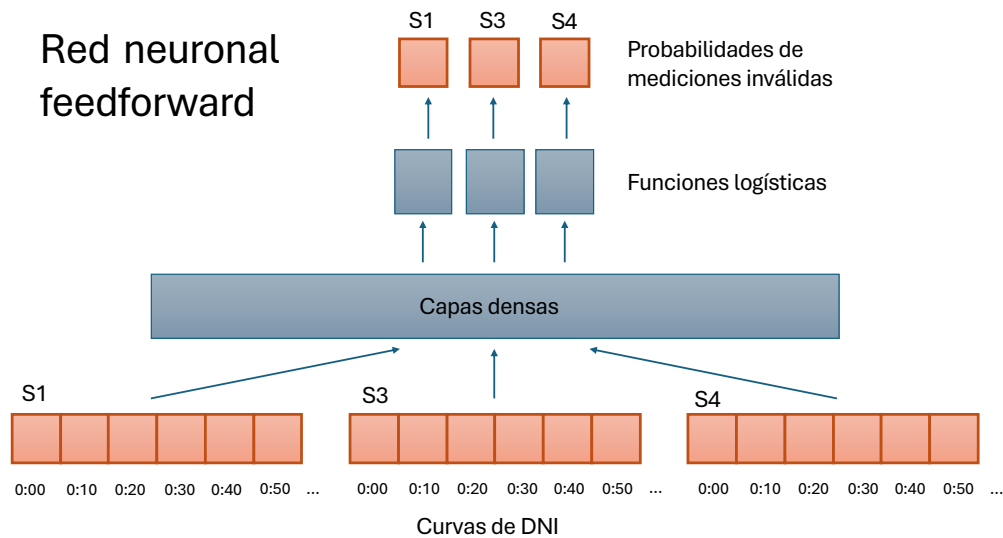


Figura 3.1: Esquema de FF

Para la entrada de este modelo se concatenan entre sí las series temporales de DNI, generando una entrada de $N_S \cdot N_M$ valores. Estos valores sirven de entrada a las capas densas de la red, que finalmente producen N_S salidas. El esquema 3.1 ilustra este modelo.

```
ff.FF(
num_sensors=NUM_S,
layer1=200,
layer2=200
)
```

Extracto de código 6: Parámetros para el modelo *feedforward* (FF)

```
class FF(nn.Module):
    def __init__(self, num_sensors, layer1, layer2):
        super(FF, self).__init__()
        self.dense1 = nn.Linear(144*num_sensors, layer1)
        self.dense2 = nn.Linear(layer1, layer2)
        self.dense3 = nn.Linear(layer2, num_sensors)
        self.name = "FF"

    def forward(self, x):
        x = rearrange(x, 'b h w -> b (h w)')
        x = F.relu(self.dense1(x))
        x = F.sigmoid(self.dense2(x))
        x = self.dense3(x)
        return x
```

Extracto de código 7: Definición del modelo de red *feedforward* (FF)

3.2. Red neuronal convolucional (CNN)

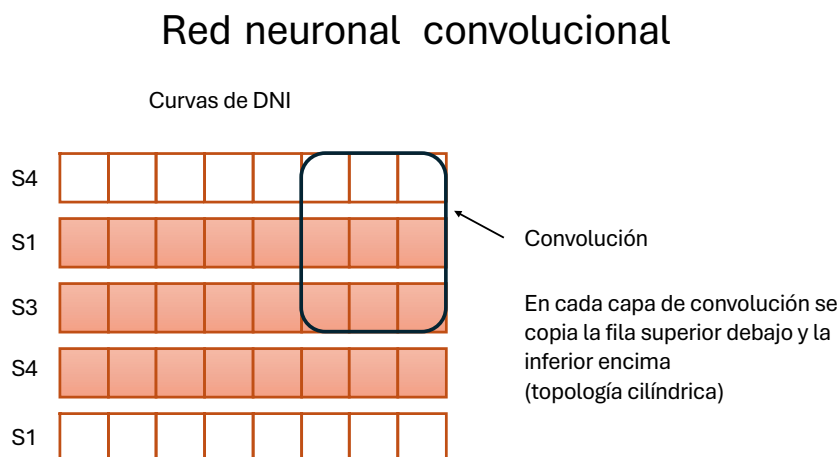


Figura 3.2: Convolución con *padding* circular

La clasificación de las curvas de radiación la realizan expertos por medio de su visualización. Es por ello que tiene sentido tratar este problema de aprendizaje profundo con modelos de visión por computador. El primero de estos modelos es una red convolucional. Este tipo de arquitectura ha demostrado muy buenos resultados en la tarea de clasificación de imágenes (Li et al. [14]).

Este modelo concatena de manera bidimensional las series temporales, formando una matriz de dimensión $N_S \times N_M$. Dado que el objetivo de las convoluciones de un modelo CNN es obtener información de los patrones existentes entre píxeles próximos de una imagen, la idea es realizar algo similar entre mediciones cercanas tanto temporalmente (eje x) como espacialmente (eje y).

Para hacer que las convoluciones tomen información de todas las series temporales se ha decidido hacerlas de N_S elementos de alto y aplicar un *padding* circular. Este *padding* consiste en copiar las $\lfloor \frac{N_S}{2} \rfloor$ series temporales tanto superiores como inferiores en el extremo opuesto. El esquema 3.2 muestra el funcionamiento de esta convolución.

Red neuronal convolucional

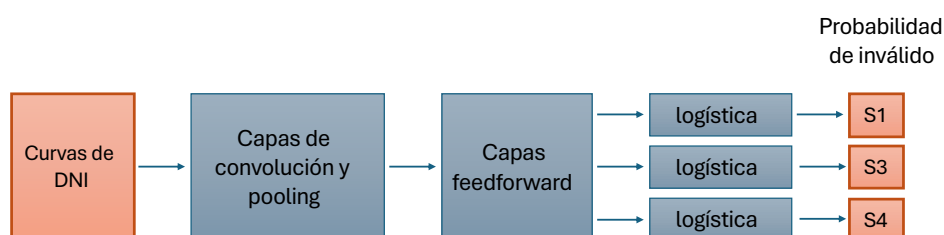


Figura 3.3: Esquema general de CNN

El esquema general que sigue este modelo es el típico de esta arquitectura (3.3). La fase de extracción de características involucra convoluciones y capas de *pooling*. Cada convolución crea nuevos canales que sirven para aumentar la complejidad del modelo. Por último, la fase de clasificación consiste en una red *feedforward*.

```
cnn.CNN(  
    num_sensors=NUM_S,  
    batch_size=batch_size,  
    channels=[4,3,2],  
    kernels=[4,4,4],  
    dense=[200,13]  
)
```

Extracto de código 8: Parámetros para el modelo CNN

```
class CNN(nn.Module):

    def __init__(self,num_sensors,batch_size,channels,kernels,dense):
        super(CNN,self).__init__()
        self.conv1 = nn.Conv2d(1,channels[0],(num_sensors,kernels[0]))
        self.conv2 = nn.Conv2d(channels[0],channels[0]*channels[1],(num_sensors,kernels[1]))
        self.conv3 = nn.Conv2d(channels[0]*channels[1],\
channels[0]*channels[1]*channels[2],(num_sensors,kernels[2]))
        self.pool = nn.MaxPool2d((1,2))
        self.flatten = nn.Flatten()
        self.batch_size = batch_size
        self.num_sensors = num_sensors
        self.name = "CNN"

        dummy_input = torch.zeros((batch_size,num_sensors, 144)) # shape: (B, H, W)
        dummy_input = rearrange(dummy_input, 'b h w -> b 1 h w')
        dummy_input = F.pad(dummy_input, (0, 0, 1, 1), mode='circular')
        dummy_input = self.pool(F.sigmoid(self.conv1(dummy_input)))
        dummy_input = F.pad(dummy_input, (0, 0, 1, 1), mode='circular')
        dummy_input = self.pool(F.sigmoid(self.conv2(dummy_input)))
        dummy_input = F.pad(dummy_input, (0, 0, 1, 1), mode='circular')
        dummy_input = self.pool(F.sigmoid(self.conv3(dummy_input)))
        flattened_size = dummy_input.view(batch_size, -1).shape[1]

        self.dense1 = nn.Linear(flattened_size,dense[0])
        self.dense2 = nn.Linear(dense[0],dense[1])
        self.dense3 = nn.Linear(dense[1],num_sensors)
        self.dropout1 = nn.Dropout(p = 0.25)
        self.dropout2 = nn.Dropout(p = 0.5)

    def forward(self, x):
        x = rearrange(x, 'b h w -> b 1 h w')
        x = F.pad(x, (0, 0, 1, 1), mode='circular')
        x = self.pool(F.relu(self.conv1(x)))
        x = self.dropout1(x)
        x = F.pad(x, (0, 0, 1, 1), mode='circular')
        x = self.pool(F.relu(self.conv2(x)))
        x = F.pad(x, (0, 0, 1, 1), mode='circular')
        x = self.pool(F.relu(self.conv3(x)))
        x = self.flatten(x)
        x = F.relu(self.dense1(x))
        x = self.dropout2(x)
        x = F.relu(self.dense2(x))
        x = self.dense3(x)
        return x
```

Extracto de código 9: Definición del modelo CNN

3.3. Transformer visual (ViT)

Siguiendo la misma estrategia que con el modelo CNN, este modelo también ha sido ideado para la clasificación de imágenes. Se trata de una implementación del algoritmo presentado en el artículo Dosovitskiy et al. [5], obtenida directamente de lucidrains [11].

Este transformer de visión supuso un hito en la tarea de clasificación de imágenes. El motivo es que es un modelo muy sencillo y que al mismo tiempo alcanza rendimientos propios del estado del arte. Para crear los embeddings del transformer únicamente hay que dividir la imagen dada en *patches* rectangulares por medio de una rejilla.

Para aplicar este procedimiento al presente ámbito, se crea de nuevo la matriz de dimensión $N_S \times N_M$. Los *patches* de la matriz son de N_S de alto, mientras que su ancho es un hiperparámetro del modelo. De esta manera, los *patches* contienen información de todos los sensores. El esquema 3.4 ilustra esta división de la matriz.

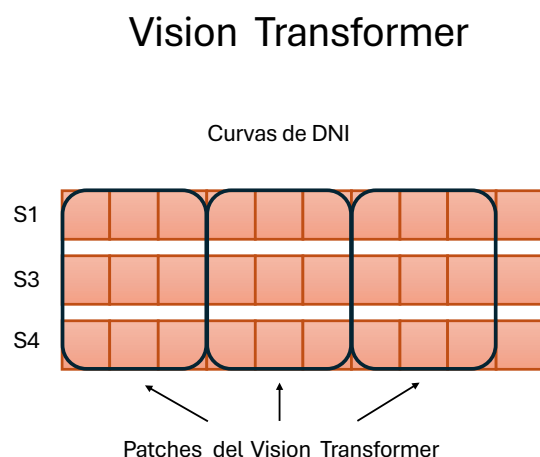


Figura 3.4: Extracción de *patches* de ViT

Una vez creados los *patches*, estos se proyectan en *embeddings*. Además, aparece un *embedding* adicional, que tendrá la función de obtener información acerca de todos los demás, de manera similar al token [class] de Devlin et al. [15]. Todos estos *embeddings* sirven de entrada al codificador transformer. Por último, la cabeza del

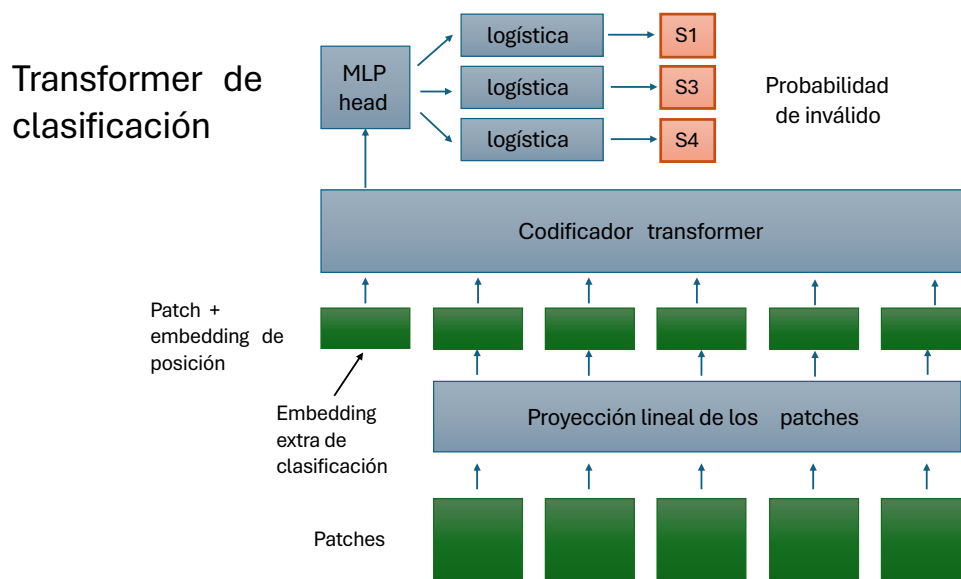


Figura 3.5: Arquitectura de un transformador clasificador

transformer se obtiene de la posición en la que se encuentra el *embedding* adicional. La arquitectura viene representada en el esquema 3.5.

El código original ha tenido que ser modificado para adaptar la forma de crear los patches al problema. El extracto de código 11 representa el código modificado.

```
transformer.ViT(
    series_length=144,
    patch_length=6,
    num_sensors=NUM_S,
    dim=12,
    depth=4,
    heads=4,
    dim_head=7,
    mlp_dim=7,
    dropout=0.1, #0.2
    emb_dropout=0.1, #0.2
)
```

Extracto de código 10: Parámetros para el modelo ViT

```
class ViT(nn.Module):
    def __init__(self, *, series_length, patch_length, num_sensors, dim,\
        depth, heads, mlp_dim, pool = 'cls', dim_head = 64, dropout = 0., emb_dropout = 0.):
        super().__init__()

        assert series_length % patch_length == 0, 'El tamaño de la serie temporal\
        debe ser divisible por el tamaño del patch.'

        num_patches = series_length // patch_length
        patch_dim = num_sensors * patch_length
        assert pool in {'cls', 'mean'}, 'pool type must be either cls (cls token)\
        or mean (mean pooling)'

        self.to_patch_embedding = nn.Sequential(
            Rearrange('b (s m) (l p) -> b l (p s m)', p = patch_length, s = num_sensors),
            nn.LayerNorm(patch_dim),
            nn.Linear(patch_dim, dim),
            nn.LayerNorm(dim),
        )

        self.pos_embedding = nn.Parameter(torch.randn(1, num_patches + 1, dim))
        self.cls_token = nn.Parameter(torch.randn(1, 1, dim))
        self.dropout = nn.Dropout(emb_dropout)
        self.sensors = num_sensors
        self.name = "ViT"

        self.transformer = Transformer(dim, depth, heads, dim_head, mlp_dim, dropout)

        self.pool = pool
        self.to_latent = nn.Identity()

        self.mlp_head = nn.Linear(dim, num_sensors)
```

Extracto de código 11: Modificación de la definición de ViT para la adaptación al problema

3.4. PatchTST

Patch time series Transformer (PatchTST) es una arquitectura transformer diseñada para la predicción de series temporales (Nie et al. [6]). Tiene dos cualidades principales:

- La segmentación de las series temporales en *patches* que sirven de input al transformer.

- Independencia de canales, con la que cada canal contiene una sola serie temporal univariable que comparte los pesos del transformer.

Aunque originalmente esté pensada para la tarea de predicción, puede ser fácilmente aplicada a la tarea de clasificación de series temporales. Para ello, solo hay que realizar una transformación lineal en la última capa del transformer con dimensión de salida igual al número de diferentes valores de clasificación.

Las propiedades de esta arquitectura la hacen muy apropiada para resolver este problema. En primer lugar, está específicamente diseñada para series temporales y, además, admite como entrada varias series al mismo tiempo. Una característica de este modelo es la total independencia entre los diferentes canales. En el artículo se afirma que esta característica aumenta el rendimiento y, al mismo tiempo, disminuye en gran medida la complejidad del modelo. Sin embargo, en este problema es necesario comparar las curvas de radiación de los diferentes sensores para poder identificar algunos de los tipos de errores de medición existentes. Pero este modelo trata cada serie temporal como un único elemento, impidiendo identificar este tipo de patrones. Aún así, es interesante explorar qué resultados otorga un modelo con independencia de los canales en contraposición al transformer de visión, en el que los *embeddings* sí que capturan información de todas las series temporales de cada sensor.

En la figura 3.6 se esquematiza la forma en la que se realiza la división de las series temporales en *patches*. Cada uno de estos son de igual tamaño, y pueden superponerse unos con otros.

Este modelo difiere en el ViT esencialmente en la creación de *patches*. El resto de la arquitectura es similar a la del esquema 3.5.

En la implementación fuente (Hugging-face [13]) se calcula la salida de la función de pérdida junto a la función sigmoide dentro del modelo. Esto se ha tenido que modificar puesto que en el programa se calcula fuera. El código 13 muestra el código modificado.

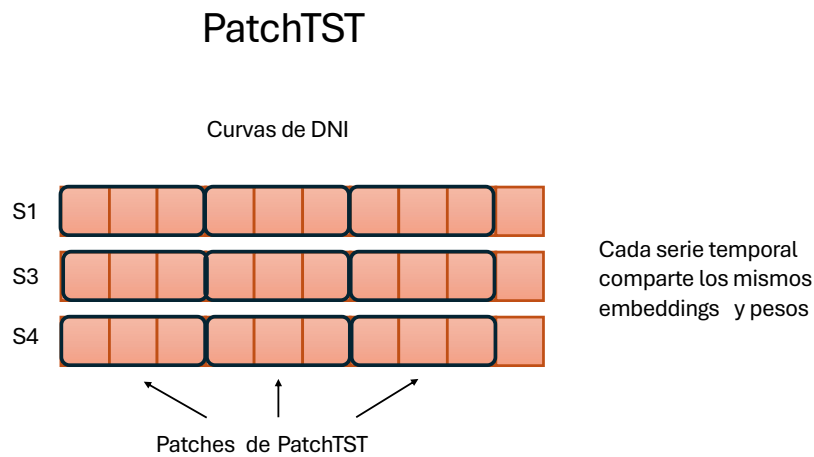


Figura 3.6: Extracción de *patches* de PatchTST

```
ptst_config.PatchTSTConfig(  
    num_input_channels=NUM_S,  
    num_targets=3,  
    d_model=12,  
    ffn_dim=102,  
    context_length=144,  
    patch_length=12,  
    patch_stride=12,  
    use_cls_token=False,  
    num_attention_heads=3,  
    num_hidden_layers=3,  
    channel_attention=True,  
    attention_dropout=0.1,  
    positional_dropout=0.1,  
)
```

Extracto de código 12: Parámetros para el modelo PTST

```
return_dict = return_dict if return_dict is not None else self.config.use_return_dict

model_output = self.model(
    past_values=past_values,
    past_observed_mask=past_observed_mask,
    output_hidden_states=output_hidden_states,
    output_attentions=output_attentions,
    return_dict=True,
)
y_hat = self.head(model_output.last_hidden_state)

return y_hat
```

Extracto de código 13: Modificación de la definición de PatchTST para la adaptación al problema

3.5. Optimización de hiperparámetros

El extracto de código 14 muestra los hiperparámetros que se han optimizado para cada modelo. Aunque se incluyan también los parámetros tasa de aprendizaje (`learning_rate`) y umbral (`threshold`), estos se han dejado en 0,001 y 0,5 respectivamente. No obstante, existe una sección en la parte de experimentación (5.2) donde se estudia cuál es el mejor valor para el umbral.

```
pbounds_ff = {'learning_rate': (0.0001, 0.001), 'threshold': (0,1),\
              'layer1': (10,1000, int), 'layer2':(10,1000,int)}
pbounds_cnn = {'learning_rate': (0.0001, 0.001),
               'threshold': (0,1),
               'channel1': (1, 5, int),
               'channel2': (1, 5, int),
               'channel3': (1, 5, int),
               'kernel1':(3,5,int),
               'kernel2':(3,5,int),
               'kernel3':(3,5,int),
               'dense1':(10,500,int),
               'dense2':(10,200,int)}

pbounds_vit = {'learning_rate': (0.0001, 0.001),
               'threshold': (0,1),
               'dim': (6,18,int),
               'depth': (2,4,int),
               'heads': (2,4,int),
               'dim_head': (4,12,int),
               'mlp_dim': (6,200,int),
               'dropout':(0,0.2),
               'emb_dropout':(0,0.2)}

pbounds_ptst = {'learning_rate': (0.0001,0.001),
                'threshold': (0,1),
                'ffn_dim': (50,200,int),
                'num_attention_heads': (2,4,int),
                'num_hidden_layers': (2,4,int),
                'attention_dropout': (0,0.2),
                'positional_dropout': (0,0.2)
                }
```

Extracto de código 14: Parámetros a optimizar con optimización bayesiana y sus intervalos de búsqueda. Un intervalo (a,b) busca valores entre a y b (inclusive). Los intervalos (a,b, int) son para variables enteras. Se incluyen los modelos FF, CNN, ViT y PTST

4. Dificultades encontradas

En este capítulo se expondrán las principales dificultades que se han encontrado a lo largo del trabajo y cómo se han abordado.

4.1. Depuración del conjunto de datos

El conjunto de datos proporcionado no había sido revisado de manera exhaustiva y presentaba numerosos errores de clasificación. Esto lo vimos al observar algunos ejemplos de las curvas de radiación. Esto era un problema, pues el conjunto de datos es imprescindible que sea de calidad para que los modelos puedan aprender correctamente y sin confundirse. Por ello, una de las primeras tareas que tuve que realizar fue apuntar a mano todos los errores que existían en el conjunto de datos. Para ello conté con la ayuda de Miguel, que me enseñó a identificar los errores de medición que pueden existir en las curvas de radiación, y también de José Luis, que entre él y yo creamos un programa para visualizar fácilmente las curvas.

Claramente esta dificultad ha sido la que más ha interferido en el trabajo. Revisar todo el conjunto de datos me llevó varias horas, hay que tener en cuenta que se trata de 2371 días. Además, para que no se me pasase ningún fallo, examiné todo el conjunto de datos tres veces, cada una fijándome en un sensor.

Esta tarea, aunque tediosa, me ha servido para familiarizarme con el conjunto de datos y aprender sobre los tipos de errores de medición que pueden existir en este ámbito.

4.2. Aprendizaje de Pytorch

Antes de empezar este trabajo no tenía experiencia con esta librería. Aún así he decidido utilizarla por la potencia y flexibilidad que aporta a la hora de diseñar modelos de aprendizaje profundo. En realidad, el proceso de aprendizaje no fue difícil, con la gran comunidad que tiene esta librería existen recursos en abundancia para tomar referencias. Además su documentación es clara y está bien detallada.

Con lo que, en definitiva, este problema ha sido de leve magnitud.

4.3. Optimización de modelos

Con el objetivo de conseguir modelos que rindiesen lo mejor posible, he dedicado tiempo a encontrar sus mejores hiperparámetros. Para ello me he valido de la librería *Bayesian Optimization*, que implementa un algoritmo de optimización bayesiana mediante procesos gaussianos. En total, me ha llevado unas 10 horas de ejecución para optimizar los parámetros de todos los algoritmos.

Antes de ello, intenté hacer funcionar el programa en la GPU con el fin de hacer las ejecuciones más eficientes y rápidas. Para ello tuve que instalar las librerías de Python necesarias, así como las herramientas CUDA de Nvidia. Después de hacerlo funcionar, observé que el rendimiento no mejoraba. Esto parece ser debido a que el tamaño del conjunto de datos y (por consiguiente) de los *batches* no es demasiado grande, con lo que una paralelización del código no proporciona una clara mejora en los tiempos de ejecución. De manera que finalmente opté por ejecutar el código en la CPU, como hasta ahora.

Este proceso, sin duda, ha merecido la pena. El modelo que más se ha beneficiado es el CNN, que ha pasado a ser quizás el de mejor rendimiento. El código 15 muestra el código de este modelo anterior a su optimización. En este caso solo tiene dos capas convolucionales. Además, no tiene *dropout*. Esto hace que se cree un gran sobreajuste al conjunto de entrenamiento, como se puede ver en la figura de las gráficas de entrenamiento 4.1. Llega prácticamente a un 100 % de acierto, con lo que no es capaz de aprender más a partir de ese momento. Otro problema es que la función de pérdida toma valores demasiado altos en el conjunto de validación.

Por otra parte, también he hecho pruebas modificando el valor del hiperparámetro de PatchTST `channel_attention`. Este parámetro, si vale `True`, hace que se calculen también valores de atención entre elementos de diferentes canales. Esto va en contra del principio de independencia de canales propuesto en el artículo de PatchTST, sin embargo la implementación de Hugging Face tiene esta función. Se ha comprobado que el rendimiento mejora con este parámetro en `True`. La gráfica 4.2 representa el entrenamiento del modelo con este parámetro desactivado. Finalmente, para el modelo final he decidido poner este parámetro en `True`.

```
class CNN_antiguo(nn.Module):

    def __init__(self,num_sensors,batch_size):
        super(CNN_antiguo,self).__init__()

        self.conv1 = nn.Conv2d(1,3,(num_sensors,3))
        self.conv2 = nn.Conv2d(3,9,(num_sensors,3))
        self.pool = nn.MaxPool2d((1,2))
        self.flatten = nn.Flatten()
        self.batch_size = batch_size
        self.num_sensors = num_sensors
        self.name = "CNN antiguo"

        # Compute the flattened feature size after convs and pooling
        dummy_input = torch.zeros((batch_size,num_sensors, 144)) # shape: (B, H, W)
        dummy_input = rearrange(dummy_input, 'b h w -> b 1 h w')
        dummy_input = F.pad(dummy_input, (0, 0, 1, 1), mode='circular')
        dummy_input = self.pool(F.sigmoid(self.conv1(dummy_input)))
        dummy_input = F.pad(dummy_input, (0, 0, 1, 1), mode='circular')
        dummy_input = self.pool(F.sigmoid(self.conv2(dummy_input)))
        flattened_size = dummy_input.view(batch_size, -1).shape[1]

        self.dense1 = nn.Linear(flattened_size,100)
        self.dense2 = nn.Linear(100,num_sensors)

    def forward(self, x):

        x = rearrange(x, 'b h w -> b 1 h w')
        x = F.pad(x, (0, 0, 1, 1), mode='circular')
        x = self.pool(F.relu(self.conv1(x)))
        x = F.pad(x, (0, 0, 1, 1), mode='circular')
        x = self.pool(F.relu(self.conv2(x)))
        x = self.flatten(x)
        x = F.relu(self.dense1(x))
        x = self.dense2(x)
        return x
```

Extracto de código 15: Código de un modelo CNN parecido al primer prototipo

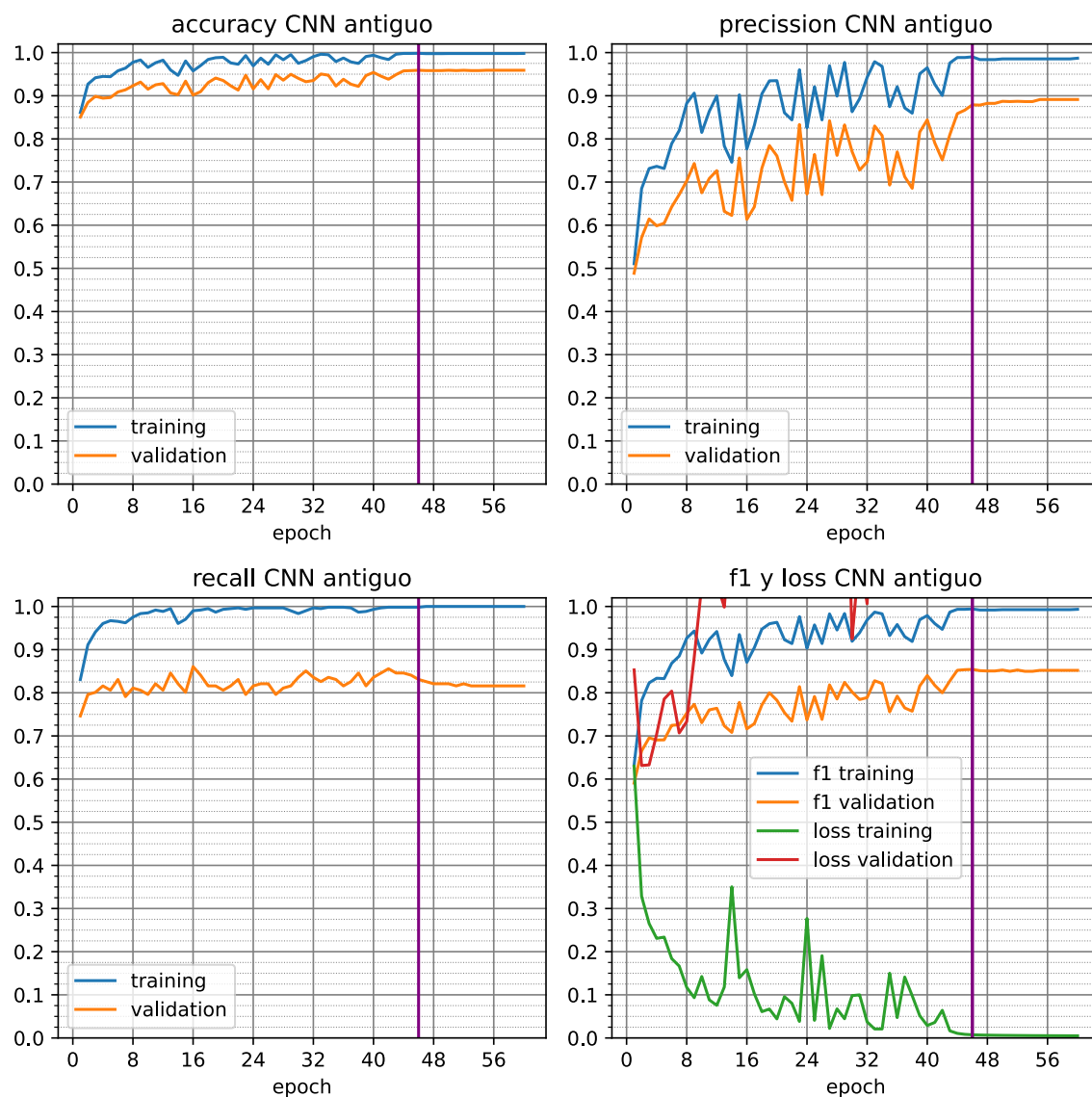


Figura 4.1: Gráficas de entrenamiento de CNN antes de optimizar

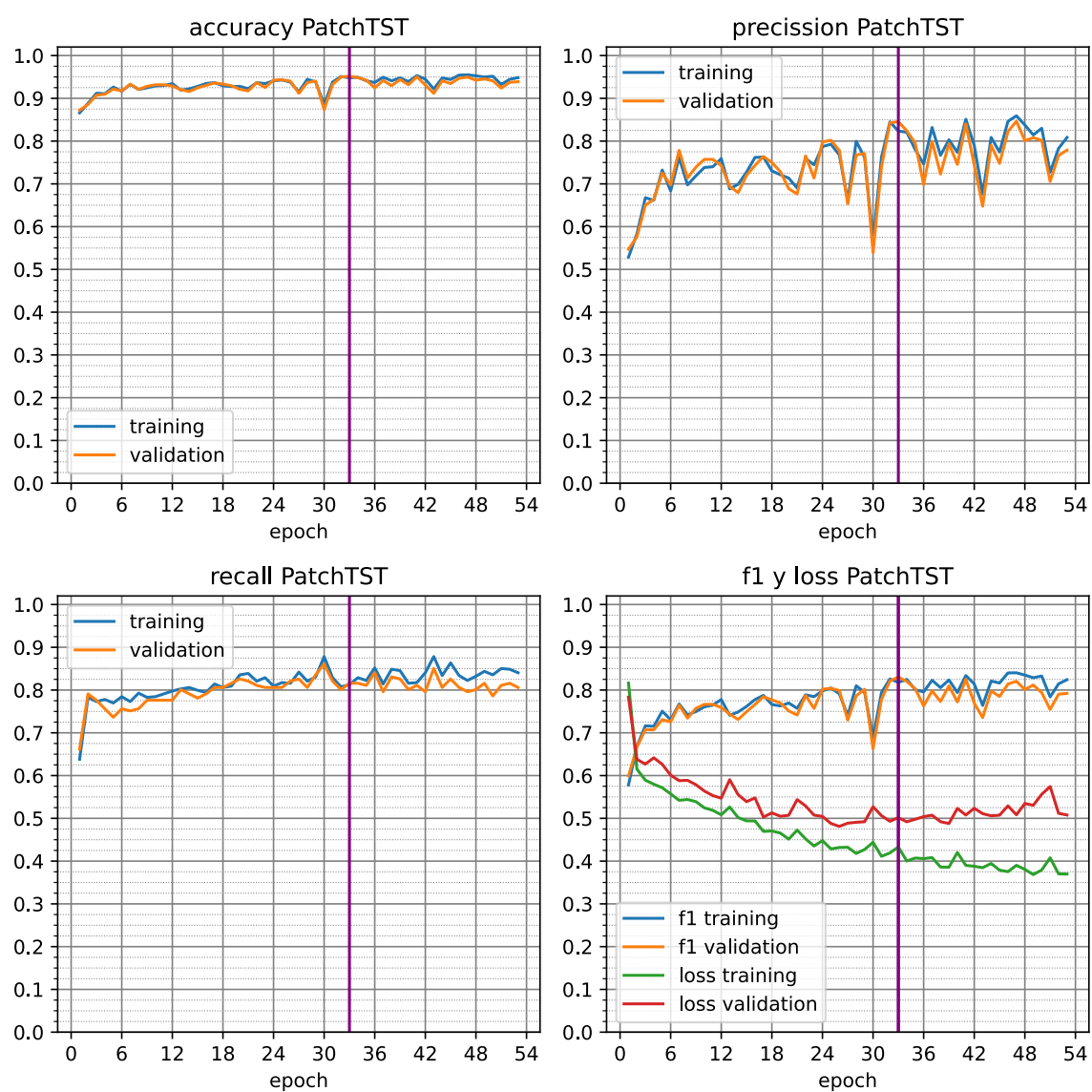


Figura 4.2: Gráficas de entrenamiento de PTST con independencia de canales

5. Experimentación

En este capítulo se expondrán los resultados obtenidos. Para empezar, se mostrará una tabla con el resumen de rendimiento de cada modelo. A continuación se hará un análisis acerca de la eficacia que cada modelo tendría en la tarea de depurar el conjunto de datos. Por último se mostrarán las gráficas obtenidas durante los entrenamientos de los modelos.

5.1. Resultados

Los modelos han sido entrenados con el conjunto de entrenamiento. El conjunto de validación ha servido para determinar el mejor conjunto de pesos a lo largo del entrenamiento. Todos los detalles acerca del proceso de entrenamiento se encuentran en la sección 2.3. La tabla 5.1 muestra las medidas de rendimiento de cada modelo: tasa de acierto (ACC), precisión (P), sensibilidad (R) y valor F1. Además, se muestra la época en la que se han conseguido los mejores pesos (epoch). Las medidas de entrenamiento se refieren a las obtenidas en el conjunto de entrenamiento en la mejor época, mientras que las medidas de prueba son las obtenidas por el conjunto de prueba utilizando los mejores pesos. Hay que recalcar que el conjunto de prueba no se ha utilizado en ningún momento durante el entrenamiento.

En la tabla se observa que el modelo CNN destaca sobre los demás, es superior en la tasa de acierto, la sensibilidad y el valor F1. Es el que consigue un mejor balance precisión-sensibilidad. El modelo ViT, por otro lado, es el que mayor valor de sensibilidad obtiene. Los modelos FF y ViT son los que peor valor F1 obtienen, aunque el balance precisión-sensibilidad es mejor en ViT.

La tabla 5.2 muestra los tiempos que ha durado el entrenamiento y la evaluación de los modelos. El tiempo de entrenamiento (T_e) se refiere únicamente a la parte de código que se ejecuta para poder actualizar los pesos: el cálculo de predicciones y de error, el cálculo de la función de coste y el algoritmo de *backpropagation*. Asimismo, el tiempo de inferencia (T_i) se refiere al cálculo de las predicciones y de las medidas de rendimiento. Por otro lado estos tiempos se han dividido entre el número de

modelo	Entrenamiento				Prueba				
	Acc	P	R	F1	Acc	P	R	F1	epoch
FF	0.953	0.917	0.743	0.821	0.946	<u>0.913</u>	0.687	0.784	37
CNN	0.988	0.937	0.982	0.959	<u>0.958</u>	0.833	<u>0.884</u>	<u>0.858</u>	45
ViT	0.975	0.865	0.979	0.918	0.934	0.725	0.864	0.788	53
PTST	0.955	0.861	0.822	0.841	0.950	0.868	0.763	0.812	22

Tabla 5.1: Tabla de resultados

Modelo	T_e (s)	T_i (s)	epochs	$T_e/\text{muestra}$ (ms)	$T_i/\text{muestra}$ (ms)
FF	59,7	71,2	57	0,74	0,66
CNN	85,4	84,2	60	1,00	0,74
ViT	131,4	98,4	60	1,54	0,86
PTST	105,1	88,3	53	1,39	0,88

Tabla 5.2: Tiempos de entrenamiento e inferencia

muestras en total para las que se ha realizado los entrenamientos y las evaluaciones:

$$T_e/\text{muestra} = \frac{T_e}{\text{epochs} * l_e} \quad (5.1)$$

$$T_i/\text{muestra} = \frac{T_i}{\text{epochs} * (l_e + l_v)} \quad (5.2)$$

El número de muestras que se procesan en la fase de entrenamiento es igual *epochs* veces la longitud del conjunto de entrenamiento (l_e), mientras que para la fase de inferencia es *epochs* veces las longitudes del conjunto de entrenamiento y de validación (l_v) porque se realiza en ambos.

5.2. Análisis de eficacia

Las medidas de rendimiento de la tabla 5.1 no son suficientes para determinar la verdadera efectividad de los modelos. Hay que recordar que el objetivo de encontrar errores de medición es eliminarlos para obtener una curva de radiación correcta que represente el día correspondiente. Esto se realiza haciendo la media de entre las series temporales válidas, por lo que debe haber al menos una válida para poder representar a la curva de radiación de ese día. Es decir, dado un día, se podrá decir que la clasificación de las curvas de radiación de los sensores ha resultado satisfactoria si se dan las siguientes condiciones:

1. Se han identificado todas las curvas de radiación inválidas. Es decir, todas las inválidas han sido marcadas como tal.
2. Al menos una curva de radiación válida ha sido identificada. Esto es, no todas las válidas han sido erróneamente marcadas como inválidas.

Estas condiciones suponen que la medida de sensibilidad es muy importante. La sensibilidad debe ser suficiente para minimizar la probabilidad de que quede una curva inválida por identificar, lo que supondría un falso negativo. En contraposición, la medida de precisión no lo es tanto. Si el modelo identifica erróneamente una curva como inválida, lo que significa un falso positivo, no tiene importancia siempre que existan otras curvas válidas correctamente identificadas.

Modelo	P	R	Días inválidos	Eficacia
FF	0,913	0,687	57	69,4 %
CNN	0,833	0,884	23	87,6 %
ViT	0,725	0,864	23	87,6 %
PTST	0,826	0,768	43	76,9 %

Tabla 5.3: Días inválidos resultantes tras eliminación de series predichas como inválidas

La tabla 5.3 muestra la cantidad de días que resultan inválidos aplicando las condiciones. La columna "Eficacia" representa el porcentaje de reducción de días inválidos respecto al número de días del conjunto de prueba en los que había al menos una serie inválida, que es igual a 186. Se puede observar que hay una correlación entre la eficacia y la sensibilidad, lo que sirve para apreciar la importancia de esta medida en este contexto.

La sensibilidad se puede aumentar disminuyendo el umbral de clasificación. Por tanto, ahora se va a analizar cómo cambia el número de días erróneos en función del umbral. Para ello se ha calculado el número de días inválidos del conjunto de validación para valores de umbral que van de 0,01 hasta 0,49, en intervalos de 0,01. El resultado obtenido se resume en la imagen 5.1.

Para cada modelo se ha tomado el umbral que ha proporcionado mayor eficacia sobre el conjunto de validación y se ha realizado el mismo cálculo con este umbral sobre el conjunto de prueba. La tabla 5.4 proporciona los resultados obtenidos.

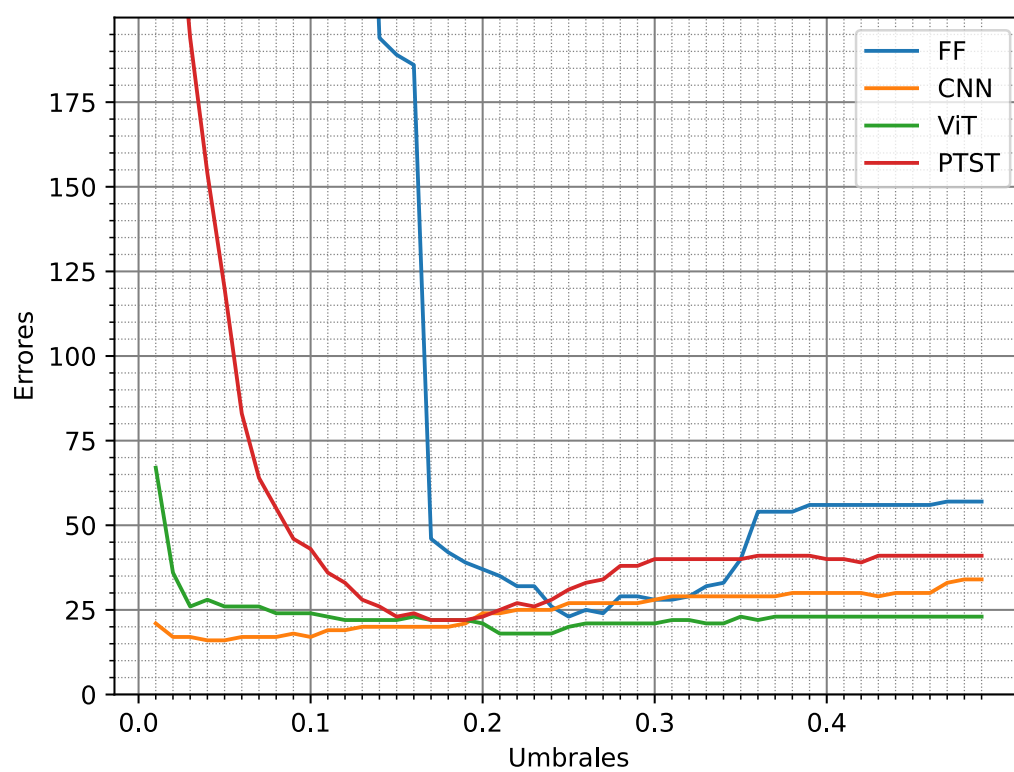


Figura 5.1: Número de días inválidos en función del umbral de clasificación

Modelo	P	R	Días inválidos	Eficacia	Umbral
FF	0.227	0.914	23	87,6 %	0,25
CNN	0.606	0.939	16	91,4 %	0,17
ViT	0.639	0.904	18	90,3 %	0,23
PTST	0.325	0.944	22	88,2 %	0,18

Tabla 5.4: Días inválidos resultantes tras eliminación de series predichas como inválidas y con umbral optimizado

Tras optimizar el umbral de clasificación, el modelo CNN se sitúa en primer lugar, llegando a una eficacia del 91,4%. Se puede observar que ha mejorado la sensibilidad sin sacrificar demasiado la precisión, quedándose en un valor aceptable para este contexto. El modelo FF sigue quedándose por debajo de los demás.

En las tablas 5.6 - 5.12 se puede ver un desglose de los errores de clasificación en el conjunto de prueba. Muestran la cantidad de falsos positivos (FP) y falsos negativos (FN) que se dan para cada configuración de etiquetas. Cada fila de las tablas representa una configuración que indica los sensores válidos (V) e inválidos (I). La columna "Total" indica la cantidad total de aparición de cada configuración en el conjunto de prueba. Seguidamente, se indican las cantidades de configuraciones para las que existen de 0 a 3 falsos positivos, y de 0 a 3 falsos negativos. Por último, la columna "Error" indica cuántos días inválidos hay para cada configuración, siguiendo los criterios anteriores. Sumando los valores de la columna "Error" se obtiene el valor correspondiente de la columna "Días inválidos" en las tablas 5.3 y 5.4.

	Total	0 FP	1 FP	2 FP	3 FP	0 FN	1 FN	2 FN	3 FN	Error
V V V	278	270	8	0	0	278	0	0	0	0
I V V	67	66	0	1	0	40	27	0	0	27
V I V	61	61	0	0	0	45	16	0	0	16
V V I	48	45	3	0	0	41	7	0	0	7
I I V	6	6	0	0	0	3	3	0	0	3
I V I	1	1	0	0	0	0	0	1	0	1
V I I	1	1	0	0	0	0	1	0	0	1
I I I	2	2	0	0	0	0	0	0	2	2

Tabla 5.5: Errores de FF con umbral 0,5

	Total	0 FP	1 FP	2 FP	3 FP	0 FN	1 FN	2 FN	3 FN	Error
V V V	278	1	43	230	4	278	0	0	0	4
I V V	67	39	27	1	0	67	0	0	0	1
V I V	61	6	53	2	0	55	6	0	0	6
V V I	48	39	4	5	0	42	6	0	0	8
I I V	6	6	0	0	0	6	0	0	0	0
I V I	1	0	1	0	0	0	1	0	0	1
V I I	1	1	0	0	0	0	1	0	0	1
I I I	2	2	0	0	0	0	1	1	0	2

Tabla 5.6: Errores de FF con umbral 0,25

	Total	0 FP	1 FP	2 FP	3 FP	0 FN	1 FN	2 FN	3 FN	Error
V V V	278	252	25	1	0	278	0	0	0	0
I V V	67	64	2	1	0	59	8	0	0	9
V I V	61	60	0	1	0	57	4	0	0	4
V V I	48	46	2	0	0	44	4	0	0	4
I I V	6	6	0	0	0	4	2	0	0	2
I V I	1	1	0	0	0	0	1	0	0	1
V I I	1	1	0	0	0	0	1	0	0	1
I I I	2	2	0	0	0	0	1	1	0	2

Tabla 5.7: Errores de CNN con umbral 0,5

	Total	0 FP	1 FP	2 FP	3 FP	0 FN	1 FN	2 FN	3 FN	Error
V V V	278	217	57	4	0	278	0	0	0	0
I V V	67	61	5	1	0	62	5	0	0	6
V I V	61	19	41	1	0	57	4	0	0	4
V V I	48	43	4	1	0	46	2	0	0	3
I I V	6	6	0	0	0	6	0	0	0	0
I V I	1	1	0	0	0	1	0	0	0	0
V I I	1	1	0	0	0	0	1	0	0	1
I I I	2	2	0	0	0	2	0	0	0	2

Tabla 5.8: Errores de CNN con umbral 0,21

	Total	0 FP	1 FP	2 FP	3 FP	0 FN	1 FN	2 FN	3 FN	Error
V V V	278	235	35	8	0	278	0	0	0	0
I V V	67	61	6	0	0	58	9	0	0	9
V I V	61	58	2	1	0	55	6	0	0	6
V V I	48	44	4	0	0	44	4	0	0	4
I I V	6	6	0	0	0	5	1	0	0	1
I V I	1	1	0	0	0	1	0	0	0	0
V I I	1	1	0	0	0	0	1	0	0	1
I I I	2	2	0	0	0	0	0	0	2	2

Tabla 5.9: Errores de ViT con umbral 0,5

	Total	0 FP	1 FP	2 FP	3 FP	0 FN	1 FN	2 FN	3 FN	Error
V V V	278	214	56	6	2	278	0	0	0	2
I V V	67	59	7	1	0	63	4	0	0	5
V I V	61	48	12	1	0	57	4	0	0	4
V V I	48	44	4	0	0	44	4	0	0	4
I I V	6	6	0	0	0	6	0	0	0	0
I V I	1	1	0	0	0	1	0	0	0	0
V I I	1	1	0	0	0	0	1	0	0	1
I I I	2	2	0	0	0	0	0	0	2	2

Tabla 5.10: Errores de ViT con umbral 0,23

	Total	0 FP	1 FP	2 FP	3 FP	0 FN	1 FN	2 FN	3 FN	Error
V V V	278	259	18	1	0	278	0	0	0	0
I V V	67	62	5	0	0	46	21	0	0	21
V I V	61	57	4	0	0	53	8	0	0	8
V V I	48	46	2	0	0	42	6	0	0	6
I I V	6	5	1	0	0	3	3	0	0	4
I V I	1	1	0	0	0	0	1	0	0	1
V I I	1	1	0	0	0	0	1	0	0	1
I I I	2	2	0	0	0	0	0	0	2	2

Tabla 5.11: Errores de PTST con umbral 0,5

	Total	0 FP	1 FP	2 FP	3 FP	0 FN	1 FN	2 FN	3 FN	Error
V V V	278	28	166	81	3	278	0	0	0	3
I V V	67	52	13	2	0	66	1	0	0	3
V I V	61	44	15	2	0	59	2	0	0	3
V V I	48	39	6	3	0	47	1	0	0	4
I I V	6	3	3	0	0	4	2	0	0	5
I V I	1	1	0	0	0	0	1	0	0	1
V I I	1	1	0	0	0	0	1	0	0	1
I I I	2	2	0	0	0	0	1	1	0	2

Tabla 5.12: Errores de PTST con umbral 0,18

Se observa como, efectivamente, en general la cantidad de falsos positivos es mayor para el umbral más bajo, mientras que la cantidad de falsos negativos es menor para el umbral más bajo.

5.3. Gráficas de entrenamiento

A continuación se muestran las gráficas de los entrenamientos de cada modelo. La mejor época aparece señalada con una recta vertical morada.

En el caso de FF se muestra un claro estancamiento de la mejoría prácticamente a partir de la época 16. La sensibilidad siempre se queda lejos de llegar al 80 %.

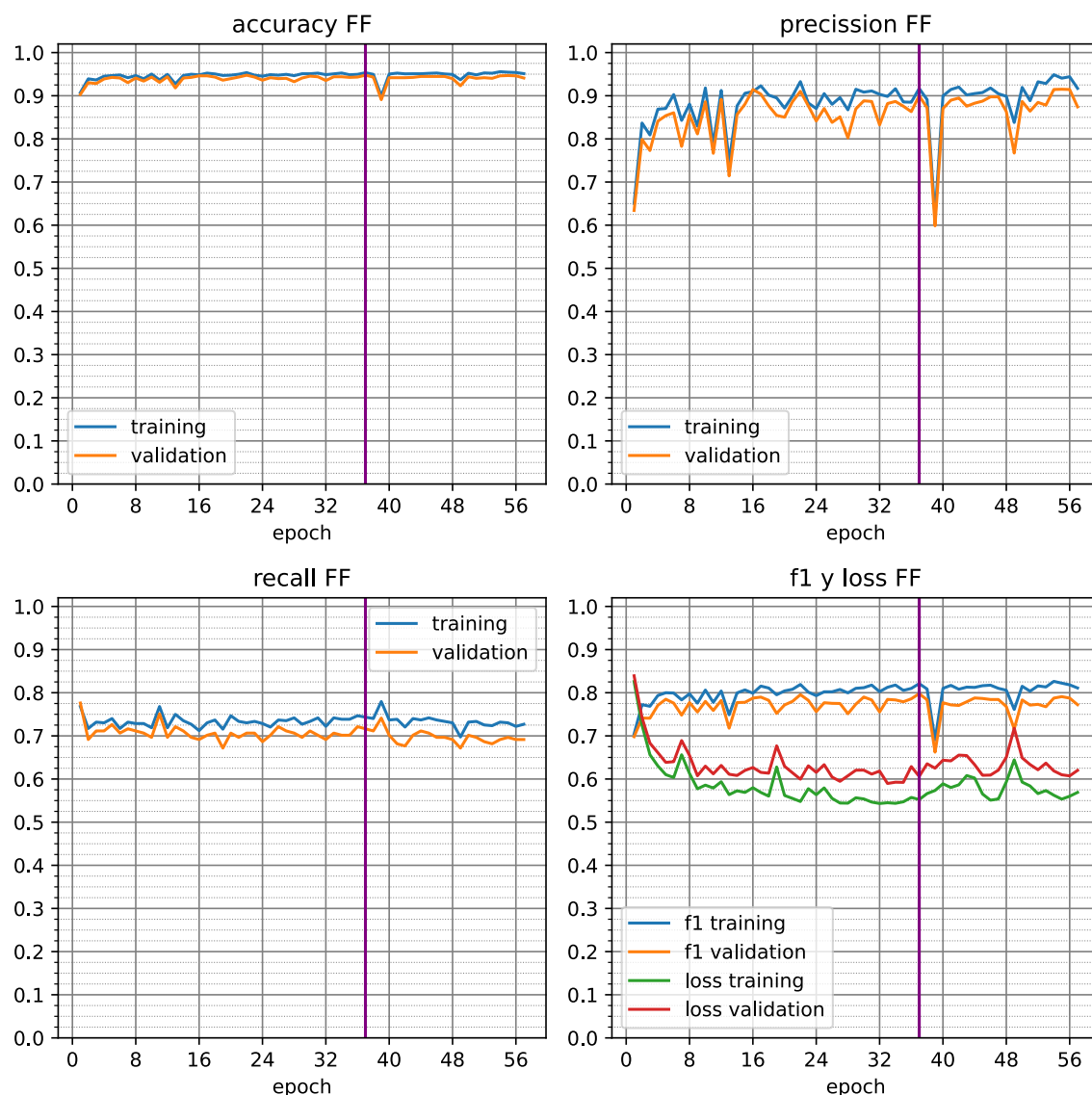


Figura 5.2: Gráficas de entrenamiento de FF

La gráfica de CNN muestra un estancamiento de la sensibilidad en torno a la época 16. Aún así el valor F1 sí mejora hasta la época 45. Por otra parte, también muestra signos de sobreajuste por la diferencia de rendimiento entre las gráficas de entrenamiento y de validación. Este sobreajuste se ha tratado de reducir mediante medidas de regularización del tipo *dropout*.

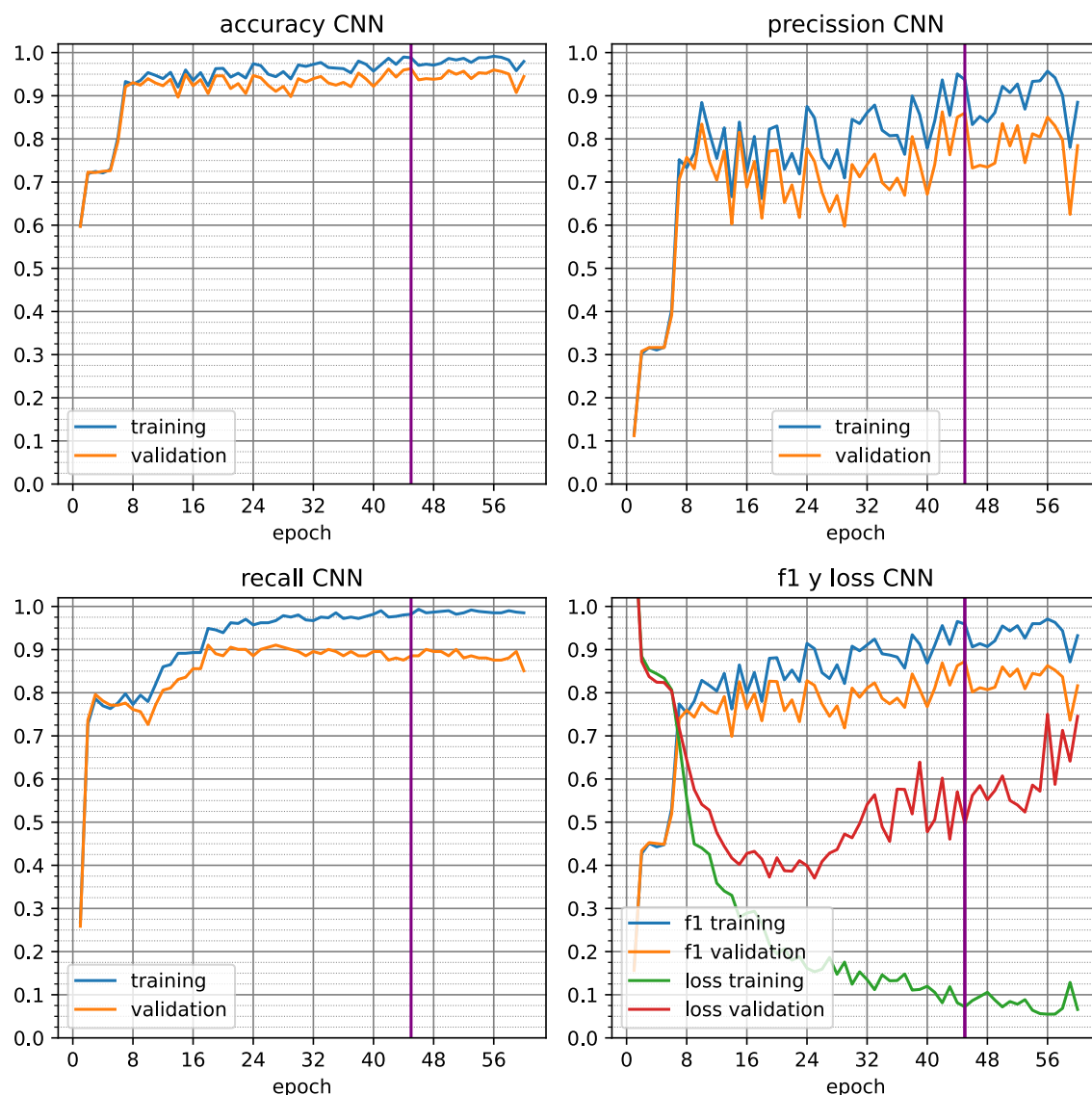


Figura 5.3: Gráficas de entrenamiento de CNN

La gráfica de ViT muestra estancamiento en el conjunto de validación mientras que el de entrenamiento sigue una tendencia ascendente, lo que significa sobreajuste. En este caso también se ha tratado de evitar mediante *dropout*. Cabe destacar la rapidez con la que alcanza valores de sensibilidad bastante altos, llegando incluso al 90 % antes de la época 24, mientras que la precisión se sitúa en un aceptable 70 %.

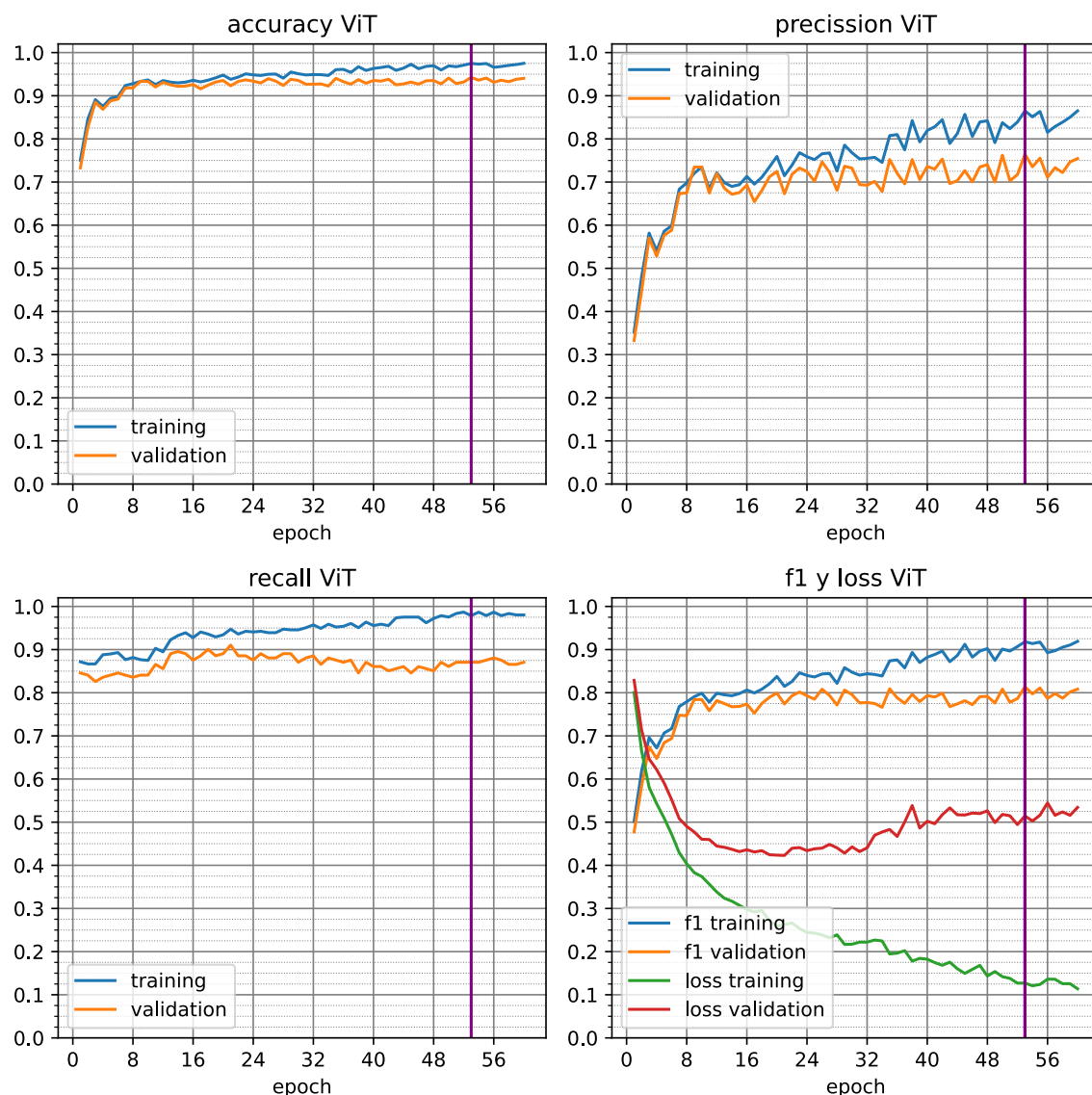


Figura 5.4: Gráficas de entrenamiento de ViT

En la gráfica de PTST se ve una tendencia ascendente leve en la métrica de sensibilidad, que se sitúa un poco por encima del 80 %. También se aprecia una escasa diferencia entre los conjuntos de entrenamiento y de validación. Esto puede ser debido a la reducción de la complejidad que implica la independencia de las series temporales.

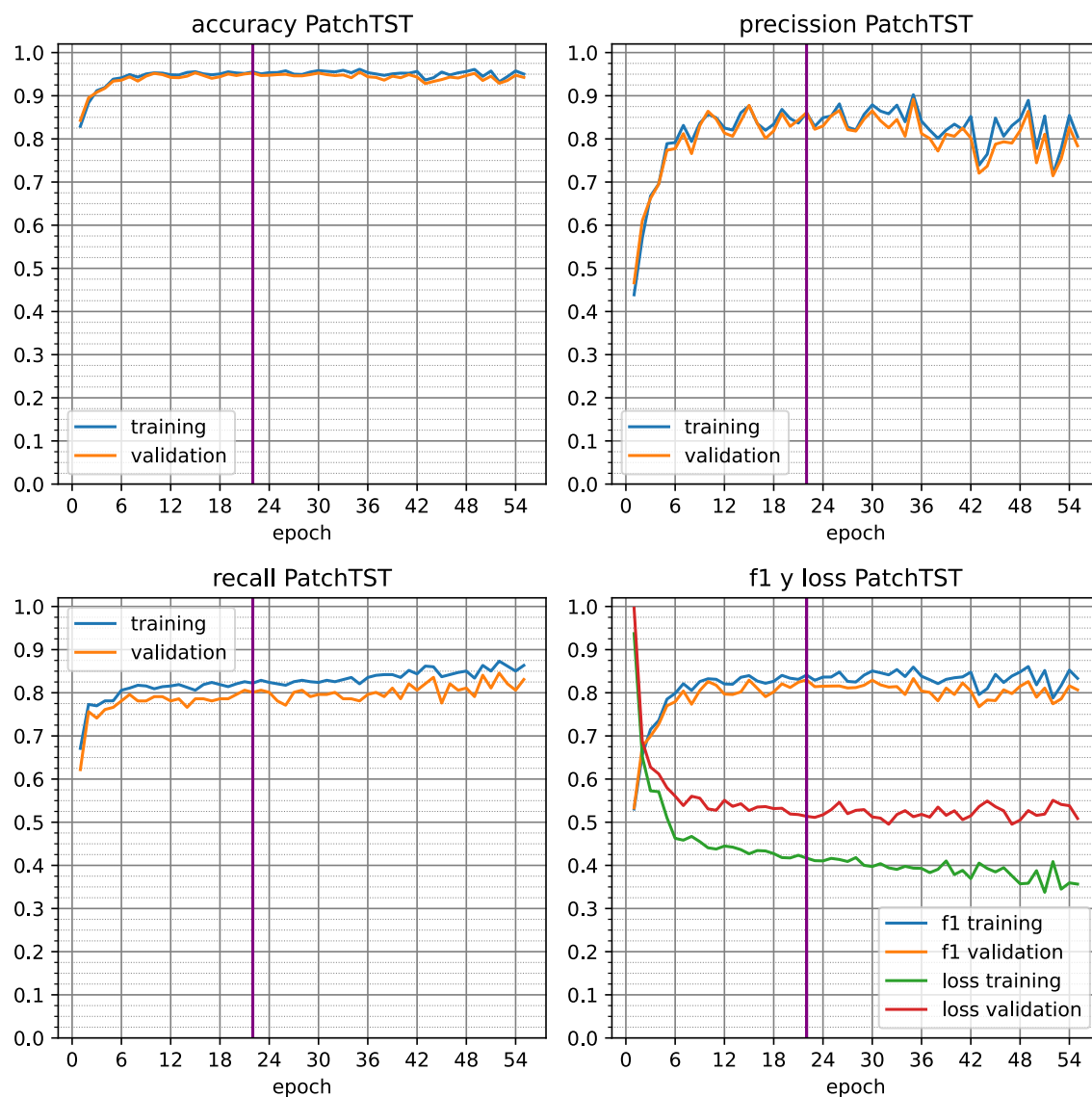


Figura 5.5: Gráficas de entrenamiento de PatchTST

6. Discusión

Esta tarea de clasificación es compleja. Existen varios tipos de fallos de medición que los sensores pueden tener, tales como suciedad, fallos eléctricos o mal alineamiento. Las curvas de radiación varían diariamente, debido al tiempo meteorológico, y estacionalmente. Todos estos elementos hacen que se den casos en los que la presencia de un fallo de medición sea difícil de determinar.

Aun así, los modelos de aprendizaje profundo han demostrado tener un buen rendimiento en esta tarea de clasificación, pudiendo llegar a alcanzar un valor F1 por encima de 0,87 en el caso de CNN. Este modelo ha podido reducir a 16 el número de días inválidos partiendo de 186 en los que al menos hay una curva inválida, lo que significa una reducción del 91,4%. Si bien es cierto que es un resultado notable, no es suficiente para reemplazar por completo al ojo humano. Sin embargo, el modelo supone una herramienta que serviría de gran ayuda para evitar fallos de clasificación.

Ha quedado demostrada la necesidad de optimizar estos modelos en favor de una mayor sensibilidad sacrificando precisión. La sensibilidad debe ser lo mayor posible, cuanto más cercana al 100 % se sitúe, mejor. Sin embargo, con un 64 % de precisión, de los 18 días inválidos que produce el modelo ViT con umbral optimizado, solo 4 se deben a falsos positivos.

El modelo CNN ha demostrado tener un mejor rendimiento sobre los transformers: produce el mayor valor F1 y es capaz de reducir en mayor medida el número de días inválidos. Esto se puede deber a que los errores de medición de radiación solar son locales. Es decir, para identificarlos basta con observar los puntos concretos donde hay error y compararlos con mediciones cercanas del mismo y de los otros sensores. La identificación de patrones locales es algo para lo que los modelos CNN dan muy buenos resultados. Por otro lado, los modelos transformers son mejores en identificar patrones globales o a largo plazo. Es por ello que, a pesar de ser los modelos más avanzados que existen en la actualidad y hayan reemplazado en muchos ámbitos a modelos más antiguos, entre los que se encuentran los basados en redes convolucionales, para este problema concreto parecen ser más eficaces estos últimos.

Por otro lado, el modelo PTST ha demostrado una eficacia superior que el modelo base, FF, sin necesidad de computar las relaciones de las tres curvas de radiación de sendos sensores, lo que demuestra la potencia de los modelos transformer.

7. Conclusiones y trabajos futuros

En este trabajo se han explorado modelos de aprendizaje profundo para la detección de errores de medición en curvas de radiación de sensores redundantes. El conjunto de datos utilizado de radiación normal directa ha tenido que ser depurado para corregir errores de clasificación. Los modelos, siguiendo diseños y optimizaciones específicamente adaptados para esta tarea, han obtenido resultados prometedores. Entre los que mejor rendimiento han logrado se encuentran la red neuronal convolucional y el transformer de visión. Aunque el primero haya obtenido un valor F1 muy superior al resto, el segundo también ha demostrado una buena eficacia resolviendo el problema principal. Ha quedado así demostrada la importancia de maximizar la sensibilidad de los modelos frente a la precisión.

Como trabajo futuro, se pueden optimizar los modelos que mejor resultados han obtenido, tratando de maximizar su sensibilidad. Asimismo, puesto que cada modelo presenta diferentes fortalezas y debilidades es una buena propuesta probar métodos *ensemble* que aprovechen las ventajas de todos los modelos simultáneamente. Por último, una funcionalidad importante que no ha sido explorada es la de reconstruir las curvas de radiación erróneas. Esto se podría conseguir, por ejemplo, con modelos transformer predictivos.

Bibliografía

- [1] Manajit Sengupta, Aron Habte, Stefan Wilbert, Christian Gueymard, Jan Remund, Elke Lorenz, Wilfried van Sark, and Adam R. Jensen. Best practices handbook for the collection and use of solar resource data for solar energy applications: Fourth edition. Technical report, International Energy Agency, 2024. <https://doi.org/10.69766/ENEH5295>.
- [2] W.D. Collins. Aerosols — role in radiative transfer. In James R. Holton, editor, *Encyclopedia of Atmospheric Sciences*, pages 48–53. Academic Press, Oxford, 2003. ISBN 978-0-12-227090-1. doi: <https://doi.org/10.1016/B0-12-227090-8/00053-1>. URL <https://www.sciencedirect.com/science/article/pii/B0122270908000531>.
- [3] Mikel Canizo, Isaac Triguero, Angel Conde, and Enrique Onieva. Multi-head cnn–rnn for multi-time series anomaly detection: An industrial case study. *Neurocomputing*, 363:246–260, 2019. ISSN 0925-2312. doi: <https://doi.org/10.1016/j.neucom.2019.07.034>. URL <https://www.sciencedirect.com/science/article/pii/S0925231219309877>.
- [4] Fan Liu, Xingshe Zhou, Jinli Cao, Zhu Wang, Tianben Wang, Hua Wang, and Yanchun Zhang. Anomaly detection in quasi-periodic time series based on automatic data segmentation and attentional lstm-cnn. *IEEE Transactions on Knowledge and Data Engineering*, 34(6):2626–2640, 2022. doi: 10.1109/TKDE.2020.3014806.
- [5] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale, 2020. URL <https://doi.org/10.48550/arXiv.2010.11929>.
- [6] Yuqi Nie, Nam H. Nguyen, Phanwadee Sinthong, and Jayant Kalagnanam. A time series is worth 64 words: Long-term forecasting with transformers, 2023. URL <https://arxiv.org/abs/2211.14730>.
- [7] Yifan Li, Mengjue Wang, Mingxiang Guan, Chen Lu, Zhiyong Li, and Tieming Chen. Saptsta-anoecg: a patchtst-based ecg anomaly detection method with

- subtractive attention and data augmentation. *Applied Intelligence*, 55(3), Dec 2024. doi: 10.1007/s10489-024-05881-5. URL <https://doi.org/10.1007/s10489-024-05881-5>.
- [8] Alex Rogozhnikov. Einops: Clear and reliable tensor manipulations with einstein-like notation. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=oapKSVM2bcj>.
- [9] Fernando Nogueira. Bayesian Optimization: Open source constrained global optimization tool for Python, 2014–. URL <https://github.com/bayesian-optimization/BayesianOptimization>.
- [10] Eduardo C Garrido-Merchán and Daniel Hernández-Lobato. Dealing with categorical and integer-valued variables in bayesian optimization with gaussian processes. *Neurocomputing*, 380:20–35, 2020.
- [11] lucidrains. vit-pytorch. URL <https://github.com/lucidrains/vit-pytorch>.
- [12] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, Joe Davison, Sam Shleifer, Patrick von Platen, Clara Ma, Yacine Jernite, Julien Plu, Canwen Xu, Teven Le Scao, Sylvain Gugger, Mariama Drame, Quentin Lhoest, and Alexander M. Rush. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, pages 38–45, Online, October 2020. Association for Computational Linguistics. URL <https://www.aclweb.org/anthology/2020.emnlp-demos.6>.
- [13] Hugging-face. Patchtst. URL https://huggingface.co/docs/transformers/v4.53.0/en/model_doc/patchtst.
- [14] Zewen Li, Fan Liu, Wenjie Yang, Shouheng Peng, and Jun Zhou. A survey of convolutional neural networks: Analysis, applications, and prospects. *IEEE Transactions on Neural Networks and Learning Systems*, 33(12):6999–7019, 2022. doi: 10.1109/TNNLS.2021.3084827.
- [15] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: pre-training of deep bidirectional transformers for language understanding. *CoRR*, abs/1810.04805, 2018. URL <http://arxiv.org/abs/1810.04805>.